PSAN4

Market Simulation for Algorithmic Trading

by

BHANDARI Om Raj
CHOY Ping Hang Gordon
KWONG Wing Fung Josh

Advised by

Prof. Pedro SANDER

Submitted in partial fulfillment of the requirements for
COMP 4981
(PSAN4)

in the

Department of Computer Science
The Hong Kong University of Science and Technology

2025-2026

Abstract

This project presents a highly modular, low-latency simulated stock exchange designed for algorithmic trading education. We systematically downscale complex, proprietary exchange architectures into an accessible and extensible C++ platform. The system features a robust, node-based Limit Order Book as its core, an intuitive Trading SDK, and a React-based visual analytics frontend. Our research includes a comparative evaluation of underlying data structures and memory management techniques. We demonstrate that custom memory pools significantly improve matching engine throughput by bypassing dynamic allocations, achieving up to a 2.61x speedup in order insertion latency. We also empirically evaluate contiguous memory layouts as experimental alternatives to the primary node-based design. The simulator supports heterogeneous agent populations and configurable market regimes, providing a realistic testbed for evaluating trading strategies and experimenting with market microstructure. Ultimately, this work delivers a comprehensive educational platform that immerses students in a fully functional mini-exchange. By enabling users to trade alongside sophisticated simulation models, the platform bridges the gap between theoretical finance classes and practical, real-world algorithmic trading skills.

Contents

1 Introduction **4**

1.1 Overview . . . 4

1.1.1 Introduction to Algorithmic Trading . . . 4

1.1.2 Need for Trading Simulation . . . 4

1.1.3 Project Contributions . . . 4

1.2 Objective . . . 5

1.3 Literature Survey . . . 6

1.3.1 Limit Order Book architecture . . . 6

1.3.2 System-level architecture . . . 7

1.3.3 Simulation methodologies . . . 7

2 Methodology **9**

2.1 Design . . . 9

2.1.1 User Access Layer: SDK & Graphical Interface . . . 9

2.1.2 External Facing API Layer: FIX Gateways . . . 10

2.1.3 Order Validation & Persistence Layer: Order Manager . . . 10

2.1.4 The Core: Matching Engine & Limit Order Book Data Structures 11

2.1.5 Distribution Layer: Market Data Processor . . . 11

2.1.6 Simulation Layer: Agents & Market Regimes . . . 11

2.2 Implementation . . . 13

2.2.1 User Access Layer: SDK & Graphical Interface . . . 14

2.2.2 External Facing API Layer: FIX Gateways . . . 17

2.2.3 Order Validation & Persistence Layer: Order Manager . . . 17

2.2.4 The Core: Matching Engine & LOB Data Structures . . . 18

2.2.5 Distribution Layer: Market Data Processor . . . 20

2.2.6 Simulation Layer: Agents & Market Regimes . . . 20

2.2.7 Server Setup and Deployment . . . 21

2.2.8 Software Engineering Practices . . . 22

2.3 Testing . . . 23

2.3.1 Test Plan and Methodology . . . 23

2.3.2 Locating Logic Errors and Bugs . . . 23

2.4 Evaluation . . . 24

2.4.1 Benchmarking . . . 24

2.4.2 Simulation Realism . . . 30

3 Discussion **34**

3.1 Project Evaluation and Reflections . . . 34

3.2 System Limitations 34

4 Conclusions and Future Work **36**

5 References **37**

6 Appendix **38**

6.1 Minutes of Project Meeting 38

6.1.1 Minutes of the 1st Project Meeting 38

6.1.2 Minutes of the 2nd Project Meeting 39

6.1.3 Minutes of the 3rd Project Meeting 40

6.1.4 Minutes of the 4th Project Meeting 41

6.1.5 Minutes of the 5th Project Meeting 42

6.1.6 Minutes of the 6th Project Meeting 43

6.1.7 Minutes of the 7th Project Meeting 44

6.1.8 Minutes of the 8th Project Meeting 45

6.1.9 Minutes of the 9th Project Meeting 46

6.1.10 Minutes of the 10th Project Meeting 47

6.1.11 Minutes of the 11th Project Meeting 48

6.1.12 Minutes of the 12th Project Meeting 49

6.1.13 Minutes of the 13th Project Meeting 50

6.1.14 Minutes of the 14th Project Meeting 51

6.2 Explanation on Limit Order Books 52

6.3 Project Planning 53

6.3.1 Distribution of Work & Gantt Chart 53

6.4 Required hardware & software 54

6.4.1 Hardware 54

6.4.2 Software 54

6.5 Comprehensive Test Cases 54

6.6 Simulation Default Configuration 58

1 Introduction

1.1 Overview

1.1.1 Introduction to Algorithmic Trading

Financial markets are complex, dynamic systems where algorithmic trading[1] plays a crucial role in price discovery[2], liquidity provision[3], and market efficiency[4]. Hong Kong, as one of the world's leading financial hubs, attracts top institutions and professionals who leverage advanced trading algorithms to remain competitive, including investment banks, hedge funds, and market makers. However, there is minimal access to high-quality, open-source algorithmic trading platforms that allow newcomers, such as university students, to interact with Limit Order Books (LOBs)[5].

1.1.2 Need for Trading Simulation

We were inspired by algorithmic trading competitions such as IMC Prosperity that demonstrate the educational value of simulated markets. Participants design systematic strategies and deploy them against unseen market data containing IMC-proprietary bots that are configured with diverse behaviours, such as market making[6]. Unfortunately, the code is closed-source and inaccessible for independent learning. This project seeks to fill the gap by creating an educational trading simulator.

1.1.3 Project Contributions

This project contributes an extensible algorithmic trading simulator and a systematic empirical study of LOB implementations, with particular emphasis on data structure design and low-latency performance characteristics in high-volume settings. The simulator recreates the environment of real-world trading by supporting multiple assets, heterogeneous agent populations, and configurable market regimes, thereby providing a controlled

[1] The use of automated computer programs to execute trades at high speeds and volumes.

[2] The process by which the market determines the fair, current price of an asset. For example, if many buyers suddenly want to purchase a stock, their competing bids drive the price up until sellers agree to match, thereby "discovering" the new higher price.

[3] The act of placing standing orders in the market (i.e., offering to buy or sell at specific quoted prices). This makes it easier for others to trade assets immediately without causing drastic price fluctuations. Think of it as providing "inventory" for the market.

[4] Driven largely by price discovery, market efficiency is the degree to which market prices quickly and accurately reflect all available information. In a highly efficient market, prices instantly adjust to new data, making it difficult to gain an unfair trading advantage.

[5] A centralized record of all outstanding orders. It acts as the core data structure of an exchange. For a detailed explanation and example, please refer to Appendix 6.2.

[6] Market makers act as liquidity providers. They continuously quote both a buy price and a sell price for an asset, profiting from the bid-ask spread while ensuring other traders can always execute orders immediately.

testbed for both strategy evaluation and microstructure experimentation.

The primary technical contribution lies in successfully "white-boxing" the traditionally opaque and highly complex architecture of public financial exchanges. While real-world exchanges operate as proprietary black boxes, this project scales these massive systems down into an accessible educational format. By synthesizing industry research and structural estimations, we engineered a transparent, functional system. Specifically, we contribute the design, implementation, and comparative evaluation of several financial exchange backends that realize the same matching logic over different underlying data structures, such as node-based containers (e.g. balanced binary search tree-style price maps with per-level linked queues) versus contiguously stored containers (e.g. vector-like arrays of price levels with in-place order queues). This enables a direct comparison of how memory layout, pointer indirection, and CPU cache locality affect core operations such as insertion, cancellation, best-quote retrieval, and full-book traversal. We also experimented with and implemented modular architectures which utilize concurrency techniques such as lock-free ring buffers.

The secondary educational contribution is the parameterizable multi-scenario configurations that expose microstructural variables to the user, such as agent order-submission intensities, inventory limits, and volatility and regime dynamics. These scenarios support different classes of agents (e.g. market makers, noise traders, and informed traders) and can be instantiated per symbol. Alongside the simulations, the project contributes a visual analytics layer that presents candlestick charts[7] for prices, recent trades, and order book depth derived from the simulated LOB state. The simulation and visualizations make the platform suitable for instructional use in courses on market microstructure[8] and algorithmic trading.

1.2 Objective

The goal of this project is to provide a safe and realistic venue for beginners in algorithmic trading to learn and test their skills. To facilitate this, we designed and implemented a robust simulated market platform for real-time trading. This platform allows users to test their trading strategies in realistic market scenarios and refine their skills through trial and error. Therefore, our objectives revolve around realistic market simulation and the ability to handle large volumes of trades, mimicking a real-world stock exchange.

The main objectives of the project were as follows:

[7] A style of financial chart used to describe price movements of a security, displaying the open, high, low, and close prices for a specific period.

[8] The study of how financial markets operate at a microscopic, tick-by-tick level. It focuses on the low-level mechanisms of trading, how different system rules affect price formation, and the specific behaviour of different populations (like retail investors vs. institutional market makers).

1. To build an efficient LOB that could handle incoming orders at the lowest latency possible.
2. To provide an intuitive method for users to submit orders.
3. To provide a clear and clean interface for visualising market information.
4. To design an efficient and robust system architecture that can handle high user traffic.
5. To implement realistic and configurable market simulation mechanisms.

For objectives 1 and 4, we conducted research on existing LOB designs, stock exchange architecture diagrams, and C++ conferences (cppcon). The researched results were then benchmarked to arrive at the most efficient solutions. To achieve objective 2, we designed and implemented a Software Development Kit (SDK) strictly in C++ to guarantee high performance and minimal latency overhead. To satisfy objective 3, we studied interfaces of commonly used brokerage applications, including IBKR and Webull. Our study allowed us to pinpoint necessary information to display and understand how data should be presented in a clear manner. For objective 4, we built a distributed architecture that can support a high volume of user-submitted orders. Finally, for objective 5, we reviewed research papers proposing theoretically plausible methods of market simulation, narrowing them down to several methods that we tested and monitored.

1.3 Literature Survey

This literature survey examines existing work on the three major components of a financial exchange: the LOB, system-level architecture, and simulation methodologies.

1.3.1 Limit Order Book architecture

The LOB sits at the core of an exchange. In real exchanges, an individual LOB for a single asset must handle message rates up to 500,000 per second with microsecond latencies [4]. Correct and efficient implementation is crucial. Essentially, it must implement add, cancel, and execute operations[9] for orders, with efficient queries such as getting the best bid and ask prices, current volume information, and the status of the order submitted by a specific user. For a detailed explanation, refer to Appendix 6.2. Current designs utilise binary trees as the primary data structure to store orders, with each node referring to a limit price. Each node then contains a doubly linked list of order nodes corresponding to those that users submit. To solve the issue of efficient queries, two hashmaps store references to limit price nodes of the tree and order nodes in the linked list, respectively [6]. However, current designs do not explicitly define which type of binary tree to use for

[9]These events are the fundamental state transitions in an exchange. "Add" places a new order into the system, "cancel" removes an existing unfilled order, and "match" occurs when a buyer's bid and a seller's ask intersect, resulting in a completed trade.

keeping the trees balanced, as the average time complexity may grow if left unmaintained. Furthermore, the literature typically only considers theoretical time complexity and lacks empirical performance metrics, such as benchmarked latency and its distributions. It also frequently ignores computer architecture considerations, such as cache locality and cache line optimization.

1.3.2 System-level architecture

Modern exchanges handle thousands of assets, each with its own LOB, and multiple concurrent users submitting orders. Across all traded assets, an exchange can face an aggregate throughput upwards of 3,000,000 messages per second at peak levels [4]. Most exchanges do not reveal their source code due to competitive advantages. Hence, existing literature only describes the architecture of exchanges at a high level [1]. Users first send messages regarding their order intentions to an exchange gateway layer, which performs validation on message integrity and runs on a separate computer from the core LOBs. This layer contains multiple parallel gateways for users to choose from and send messages to. To handle simultaneous messages and introduce a level of randomness to message handling, a sequencer first materialises the incoming messages into a single stream of ordered messages. This stream is then passed on to the LOBs for order matching, addition, or cancellation. However, current literature does not concretely define the implementation details of this architecture, such as the message and network connection protocols, load-balancing mechanisms, and inter-service communication.

1.3.3 Simulation methodologies

It is often useful to include order simulation to model behaviours of market participants. Existing open-source exchange simulation tools provide historical market data replay mechanisms and simple random order generation to model noise [5]. However, these simulators often fall short for interactive educational purposes: they are typically implemented in slower, high-level languages like Python rather than low-latency C++, and they critically lack a real-time graphical user interface for visualizing market depth and live trading.

By contrast, quantitative finance literature provides rigorous mathematical frameworks for LOB simulation. For instance, the Avellaneda-Stoikov model formally describes optimal quoting strategies for market makers managing inventory risk, while Geometric Brownian Motion (GBM) is a standard continuous-time stochastic process used to model fundamental asset price trajectories [2]. While these models provide strong theoretical foundations, the literature does not provide the robust source code necessary to implement such processes in a live, high-frequency stock exchange program.

The surveyed literature and existing platforms demonstrated useful designs for the core components of our exchange project. However, most works remained either too high-level in terms of implementation or lacked the performance and user-facing interactivity required for a modern sandbox. Our project aimed to integrate all three of these component designs to implement an efficient, C++-based, and scalable platform that is highly educational, visual, and supports advanced mathematical simulation methods.

2 Methodology

2.1 Design

The system architecture of the simulated exchange is engineered as a distributed, modular pipeline, closely mirroring the microstructure of real-world financial exchanges. It is logically partitioned into distinct microservices: user access, message validation, state persistence, core order matching, and market data distribution. The data flow follows a strict, unidirectional path. User order requests enter through the External Facing API Layer, undergo semantic validation and database logging in the Order Manager, and are synchronously routed to the Matching Engine. The engine executes the order matching algorithms and outputs raw trade events. These events are immediately consumed by the Market Data Processor via a shared-memory ring buffer, transformed into aggregated market data, and broadcast to connected clients. This decoupled architecture intentionally isolates the latency-critical matching core from network and database I/O overhead, enabling horizontal scalability under high-volume algorithmic trading scenarios.

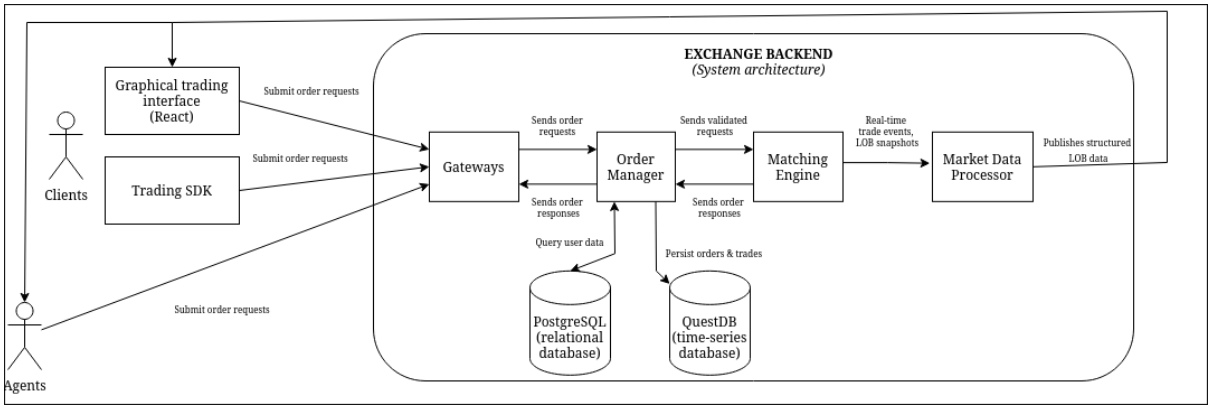

Figure 1: High-level system architecture of the exchange backend.

2.1.1 User Access Layer: SDK & Graphical Interface

To provide a seamless experience for users, the system exposes two primary interaction points:

- **Trading SDK:** We developed a programmatic interface that abstracts complex networking logic into simple, intuitive commands for order submission and cancellation. This SDK acts as a high-performance bridge between algorithmic strategies and the trading system. To handle real-time execution reports[10], the SDK provides

[10] An automated system message confirming the status of an order. For instance, if a user's order to buy 100 shares is fulfilled by the exchange, the system replies with an execution report detailing the matched price and quantity.

an event-driven architecture where users can define custom callback handlers to react to market updates seamlessly.

- **Graphical Trading Interface:** We designed a web-based dashboard inspired by professional brokerages. It serves as the visual analytics layer, rendering real-time market depth and price action. Crucially, this interface also allows users to instantiate their own customized, isolated trading environments. Analogous to "game servers" in multiplayer gaming, each user can create a server that hosts an isolated market with its own distinct set of tickers. This provisioning is orchestrated by the frontend communicating with a backend management service and a deployment manager, ensuring complete sandbox isolation for algorithmic experimentation.

2.1.2 External Facing API Layer: FIX Gateways

Mirroring real-world exchange architecture, all incoming user requests first hit the gateways to undergo simple message validation before being parsed into internal representations. This layer acts as a protocol translator, handling session management (logins/logouts) and ensuring that incoming requests are syntactically correct. By decoupling connection handling from the core matching logic, the system can scale horizontally to handle high user traffic. For instance, additional gateway instances can be dynamically spawned across different machines when demand spikes, without slowing down the core matching engine.

2.1.3 Order Validation & Persistence Layer: Order Manager

Once a request passes the gateway, it moves to the Order Manager, which serves as the central hub of the backend. It is responsible for semantic validation and data persistence:

- **Semantic Validation:** The manager performs critical checks that the gateway cannot, such as balance checking (verifying sufficient funds for a buy order) and position tracking (verifying sufficient inventory for a sell order).
- **Database Integration:** It interfaces with two distinct databases: a relational database for user profiles, credentials, and account balances; and a time-series database for high-frequency order data. Every order and trade event is persisted to ensure a robust audit trail and to allow for historical simulation replays.

After validation, the manager synchronously routes requests to the matching engine and subsequently passes confirmations back to the gateway to notify the originating user.

2.1.4 The Core: Matching Engine & Limit Order Book Data Structures

This layer is the performance-critical heart of the exchange. The engine receives a validated request, attempts to match it against the opposite side of the LOB, generates trade events if a match occurs, and updates the LOB state.

- **Matching Logic:** We utilized the Price-Time Priority algorithm, meaning orders with the best price are matched first, whereas orders at the same price level are executed on a first-come-first-served basis. This ensures that matching an incoming order against k resting orders operates with $\mathcal{O}(k)$ time complexity, as it simply traverses the head of the respective queues.
- **Data Structure Design:** We designed a theoretically optimal LOB based on algorithmic time complexity. The bid and ask sides are represented as balanced Binary Search Trees (BSTs) to provide $\mathcal{O}(\log N)$ access to price levels. Each price level contains a doubly-linked list of orders to support $\mathcal{O}(1)$ additions at the back of the queue, preserving Price-Time Priority. This is complemented by a hash map that stores iterators to every active order, enabling $\mathcal{O}(1)$ lookups and arbitrary order cancellations without traversing the book.

2.1.5 Distribution Layer: Market Data Processor

The output from the Matching Engine is raw and high-velocity. The Market Data Processor acts as the system's publisher, transforming core trade events and LOB updates into accessible formats and broadcasting them to external clients.

To maintain the performance gains achieved by the core engine, the inter-process communication (IPC) channel between the engine and the processor relies on a shared memory mechanism. This architectural choice guarantees that the matching engine is completely decoupled from the unpredictable latency introduced by string serialization and network broadcasts. By strictly delegating these computationally intensive I/O tasks to a separate process, the core engine can continue matching orders unimpeded.

2.1.6 Simulation Layer: Agents & Market Regimes

To provide a realistic and educational venue, the system includes an agent layer that simulates heterogeneous market participants interacting with the gateways just like human clients. We designed three distinct agent-based models to represent different market microstructure participants:

- **Noise Traders:** Operating as a zero-intelligence model, these agents simulate uninformed retail flow[11]. Their order arrivals are governed by a Poisson process,

[11] Orders coming from individual, non-professional investors. "Uninformed" implies these trades are

meaning the inter-arrival time Δt between consecutive orders follows an exponential distribution:

$$f(\Delta t; \lambda) = \lambda e^{-\lambda \Delta t} \tag{1}$$

where λ represents the order intensity (events per second). Order parameters are generated using randomized statistical distributions to provide baseline market activity. Specifically, the side of the order (buy or sell) is determined by a Bernoulli distribution, and the order quantity follows a Pareto distribution, defined by its scale and shape parameters, resulting in heavy-tailed order sizes. The agents are initialized with a configured number of active noise traders and a starting asset inventory.

- **Market Makers:** Implementing the Avellaneda-Stoikov model, these agents provide liquidity by maintaining a ladder of quotes around the spread. They actively manage inventory risk by calculating a reservation price r (the theoretical mid-price adjusted for inventory) and an optimal spread δ:

$$r = s - q\gamma\sigma^2(T - t) \tag{2}$$

$$\delta = \gamma\sigma^2(T - t) + \frac{2}{\gamma} \ln\left(1 + \frac{\gamma}{k}\right) \tag{3}$$

  Here, s is the current market mid-price, q is the agent's inventory, γ is the risk aversion parameter, σ^2 is the asset's price variance, $T - t$ is the time horizon, and k represents the order book density. The optimal bid and ask quotes are then symmetrically placed at $r - \delta/2$ and $r + \delta/2$. Key configurable parameters include the standard order lot size, the inventory risk aversion factor, the order book liquidity density, the operational time horizon, the initial asset holdings, and the total number of market makers deployed.

- **Informed Traders & Oracle Service:** These agents act on a "true" fundamental asset price V_t provided by an external Oracle Service. When instantiated, both the Oracle Service and the simulated markets assume an initial starting fundamental asset price of \$100.0, providing a standardized baseline for subsequent price discovery and stochastic movements. The oracle generates this true price path using a Geometric Brownian Motion (GBM) augmented with Jump Diffusion to simulate sudden exogenous shocks:

$$dV_t = \mu V_t dt + \sigma V_t dW_t + V_t dJ_t \tag{4}$$

  where μ is the drift, σ is the volatility, W_t is a standard Wiener process, and J_t represents the jump process, modeled as a compound Poisson process with a

not based on private or advanced analytical information.

specified jump intensity λ and a random jump size distribution (defined by a jump mean and standard deviation). This jump term explicitly allows the model to simulate sudden, discontinuous price changes driven by exogenous shocks, such as breaking macroeconomic news or earnings reports. The Oracle service also specifies a broadcast update interval to push these price updates. The Informed Trader continuously polls the order book and the oracle. If it detects a mispricing greater than a threshold ϵ (e.g., $V_t > P_{ask} + \epsilon$), it executes aggressive snipe orders against lagging market maker quotes using a predefined aggressive order quantity. This introduces adverse selection[12] and efficiently drives the simulated market towards price discovery. The informed trader is also constrained by an initial inventory and a maximum allowed inventory limit.

For the purpose of this educational sandbox, the mathematical parameters governing these agents (e.g., the risk aversion γ, drift μ, and volatility σ) are initialized via trial and error rather than being strictly calibrated to historical market data. However, they are fully configurable via the frontend interface and deployment configuration files, allowing users to experiment with diverse market regimes such as high-volatility environments or directional trend markets.

2.2 Implementation

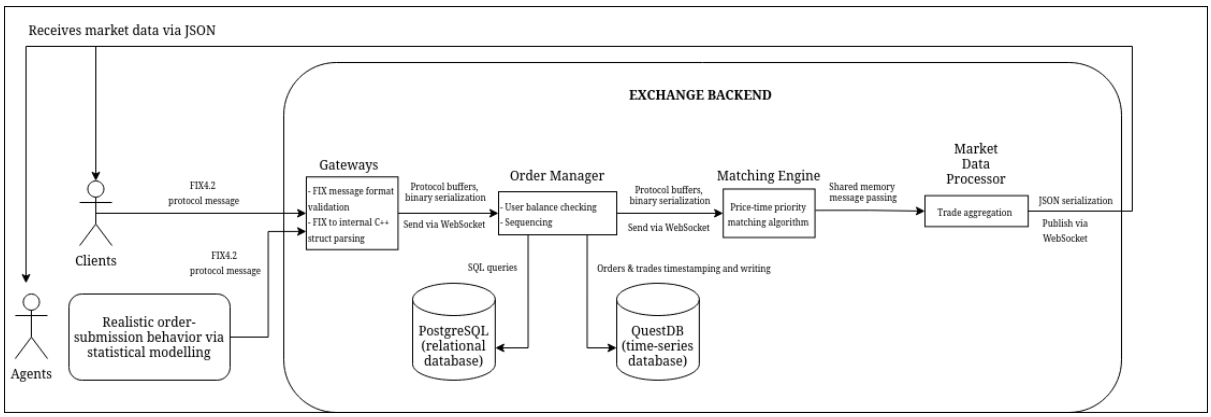

Figure 2: Detailed technical implementation of the exchange backend components.

Our implementation leverages modern C++23, prioritizing low-latency performance and robust type safety. The project uses CMake as its primary build system, facilitating dependency management and modular compilation of the various microservices. External dependencies such as QuickFIX, Boost, Protocol Buffers, and various database clients are integrated directly into the CMake workflow.

[12] A scenario where one party has better information than the other. In trading, a liquidity provider suffers adverse selection if they unknowingly sell an asset to an informed algorithm right before the asset's true value spikes.

To facilitate robust and efficient network communication, we developed a dedicated `transport` library. For structured message serialization, we utilize Google's Protocol Buffers strictly for internal, inter-service communication (such as routing validated orders from the gateway to the order manager). Their compact binary representation and rapid serialization speeds make them ideal for maintaining a low-latency trading pipeline, whereas text-based formats like JSON are reserved primarily for external communication with the frontend and public APIs. Furthermore, we implemented a custom internal WebSocket library wrapping `websocketpp`. For architectural convenience, we standardized on WebSockets for almost all internal inter-service communication (with the notable exception of the Matching Engine to Market Data Processor link, which uses a shared-memory ring buffer for ultra-low latency). This same WebSocket infrastructure is seamlessly reused to disseminate real-time market data feeds to the public React frontend.

2.2.1 User Access Layer: SDK & Graphical Interface

The client SDK was developed entirely in C++ to provide developers with a fast and intuitive interface for connecting to the trading system. It exposes a `FixClient` class that developers can subclass to interact with the market. Key methods include `submit_limit_order`, `submit_market_order`, and `cancel_order`, alongside connection management functions. To handle asynchronous execution reports and cancel rejections, the SDK provides virtual callback methods (`on_order_update` and `on_order_cancel_rejected`) that users can override to implement custom event-driven trading logic.

For the graphical trading interface, we built a responsive single-page application (SPA) using React (`frontend-v2/src`). The frontend consumes the real-time WebSocket streams from the backend and leverages charting libraries such as `lightweight-charts` and `recharts` to dynamically render interactive candlestick charts, price action, and order book depth. Furthermore, the frontend provides a control interface for users to create and manage their isolated market servers. This functionality is achieved by communicating with a backend REST service (`services/backend_service`) that orchestrates the provisioning of new environments via a centralized Python deployment script (`deployment_server.py`), allowing each user to effortlessly spin up their own independent market "game server." Figures 3 to 5 showcase screenshots of the frontend visual dashboard.

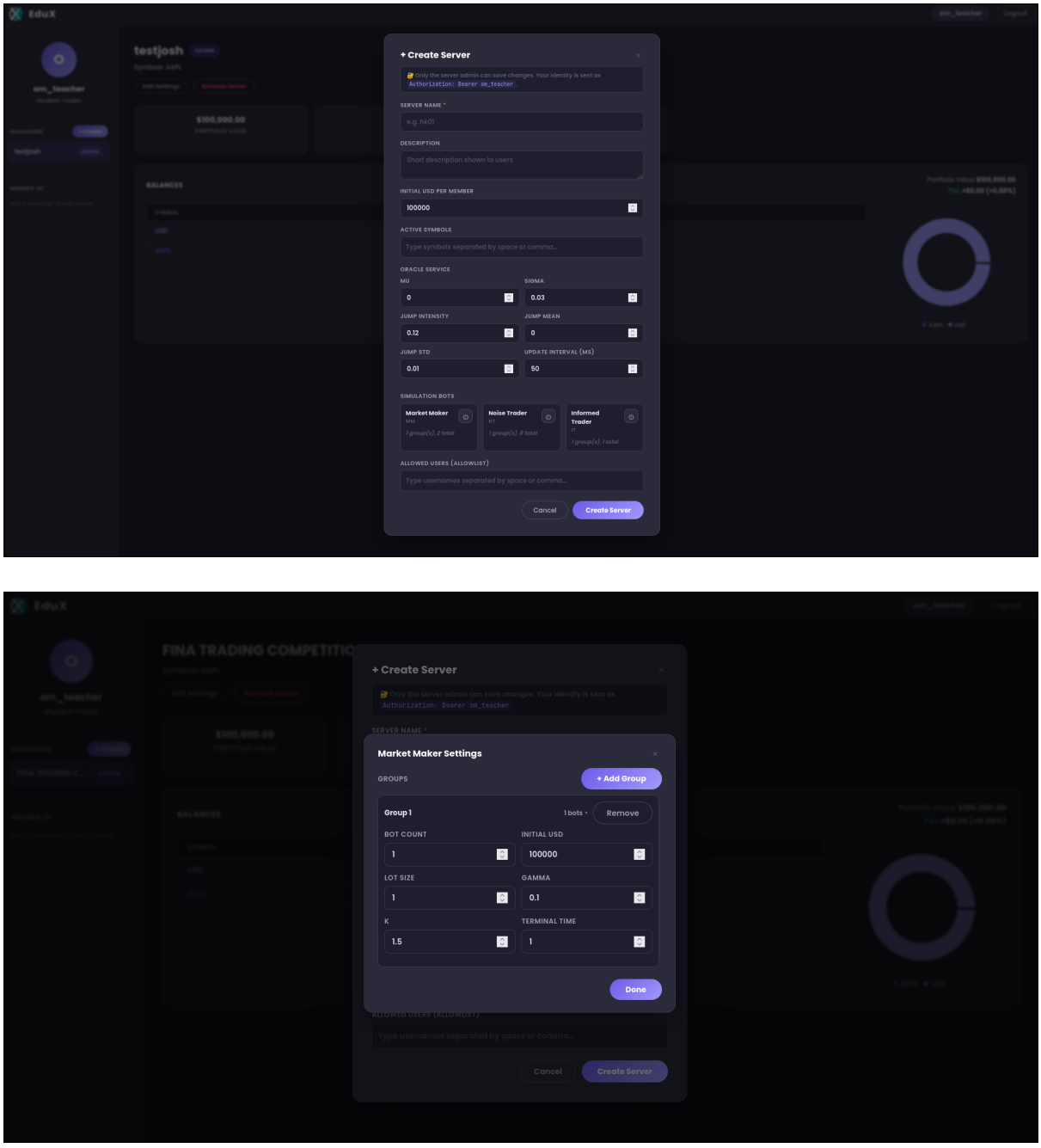

Figure 3: Server configuration and deployment interface.

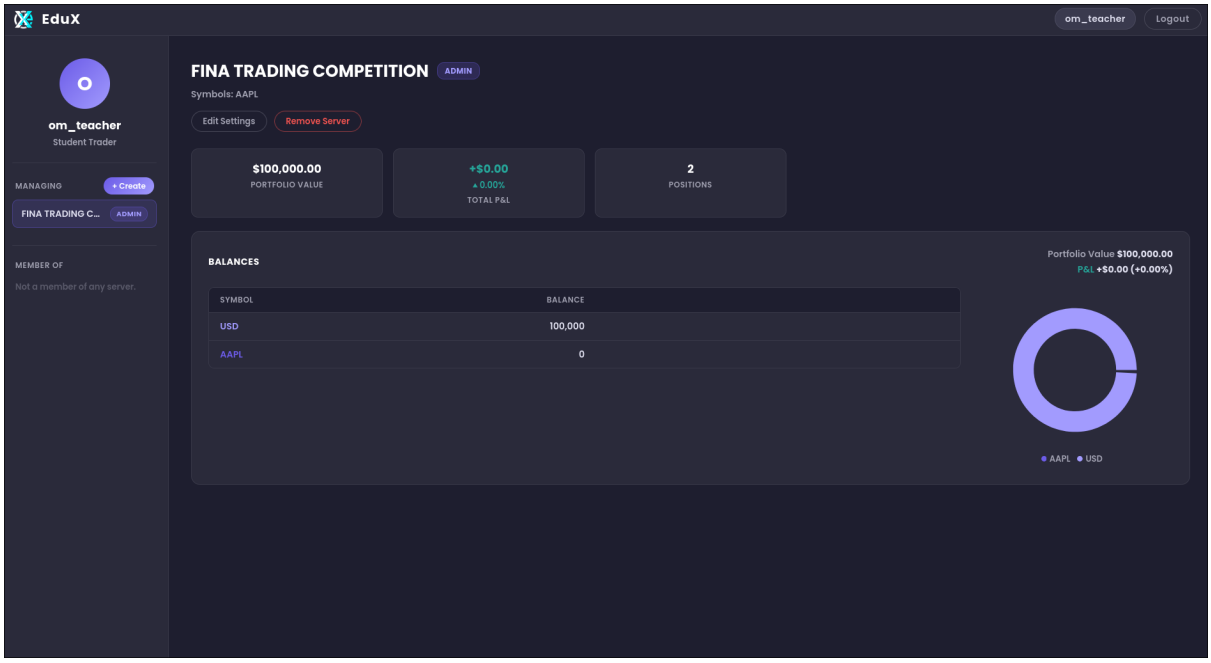

Figure 4: User portfolio and order management dashboard.

Figure 5: Real-time K-line charting and market depth visualization.

2.2.2 External Facing API Layer: FIX Gateways

The FIX Gateway was implemented using the QuickFIX C++ engine. It serves as the external-facing API, parsing incoming FIX protocol messages and translating them into internal data structures. By leveraging QuickFIX, the gateway natively handles session management, message sequencing, and heartbeat mechanisms, ensuring reliable communication with connected clients.

2.2.3 Order Validation & Persistence Layer: Order Manager

The Order Manager acts as a synchronous validator and router. To eliminate race conditions such as a single user submitting two massive orders simultaneously, the manager processes incoming messages sequentially within a single-threaded event loop. This guarantees that balance constraints and position limits are checked and updated atomically before any request is forwarded to the Matching Engine. To ensure durability without blocking this critical trading path, the persistence layer utilizes a dual-database architecture managed by a custom `DatabaseClient`.

Relational data, such as account states and user configurations, is managed using a PostgreSQL database accessed via the `libpqxx` client. Conversely, high-throughput market events and tick data are logged to a QuestDB time-series database accessed via the `c-questdb-client` library. To prevent database I/O from bottlenecking the system, we implemented an `AsyncWriter` for QuestDB ingestion. This module offloads database writes to a dedicated background thread, buffering operations like order insertions and trade logs into a thread-safe queue. The background thread batches these tasks and flushes them to QuestDB over the Ingress Line Protocol based on configurable thresholds and intervals. While validation and matching engine forwarding are synchronous to ensure correctness, this asynchronous persistence guarantees that database latency does not degrade order processing times.

To illustrate the persistence schema, the time-series definitions for order logs and trade logs in QuestDB are summarized in Tables 1 and 2. They are partitioned by day and deduplicated using UPSERT keys, which allows robust, high-throughput historical reconstruction.

Field Name	Data Type
ts	TIMESTAMP
order_id	INT
cl_order_id	INT
sender_comp_id	VARCHAR
symbol	SYMBOL
side	SYMBOL
order_qty	INT
filled_qty	INT
ord_type	SYMBOL
price	INT
time_in_force	SYMBOL
order_status	SYMBOL

Table 1: QuestDB Schema for the Orders Table.

Field Name	Data Type
ts	TIMESTAMP
trade_id	VARCHAR
price	INT
quantity	INT
symbol	SYMBOL
taker_id	VARCHAR
maker_id	VARCHAR
taker_order_id	INT
maker_order_id	INT
is_taker_buyer	BOOLEAN

Table 2: QuestDB Schema for the Trades Table.

2.2.4 The Core: Matching Engine & LOB Data Structures

At the core of the system is the Matching Engine, which manages the LOB. The LOB was implemented using C++ standard library node-based containers to achieve the theoretically optimal time complexities. Specifically, `std::map` maintains the sorted price levels; as it is typically implemented as a Red-Black Tree, it provides inherent self-balancing properties that guarantee $\mathcal{O}(\log N)$ worst-case time complexity for price level insertions and deletions, directly addressing the balancing requirements that are often overlooked in existing literature. Furthermore, `std::list` maintains the doubly-linked queue of orders at each price level, and `std::unordered_map` provides constant-time lookups to the list iterators for instant order cancellations. To communicate executions and market state changes to downstream services, the matching engine writes to a custom Multiple-Producer Single-Consumer (MPSC) lock-free ring buffer. This ring buffer is backed by POSIX shared memory (`shm_open` and `mmap`).

To achieve optimal IPC throughput, the buffer capacity was constrained to a power of two, replacing expensive modulo operations with bitwise AND operations for index wrapping. These constant-time bitwise and atomic operations ensure that both pushing and popping events operate with $\mathcal{O}(1)$ time complexity. Additionally, to prevent false sharing, the internal structures incorporate explicit memory alignment and padding (`alignas(64)` for x86 architectures), ensuring producer and consumer indices reside on separate CPU cache

lines. In the edge case where the Market Data Processor (Consumer) falls behind the Matching Engine (Producer), the lock-free ring buffer will report as full. Currently, the engine utilizes a non-blocking `try_push` mechanism (as seen in the `try_publish` calls); if the buffer is at capacity, new events are dropped to prevent the core matching logic from blocking, emphasizing the need for appropriately sized buffers and fast consumer polling.

Listing 1 demonstrates the critical path logic for crossing an incoming order against resting liquidity. The algorithm sequentially walks the best available price levels, decrementing quantities and generating `Trade` events until the incoming order is either fully filled or the opposing liquidity at marketable prices is exhausted, successfully achieving the target $\mathcal{O}(k)$ matching time complexity defined in our architecture design.

```
void LimitOrderBook::match_order(SideContainer& near_side,
    SideContainer& far_side, int price, int remaining_quantity, int
    order_id, Side side, std::string_view broker_id) {
     // 1. Iterate through the opposing side of the book as long as we
        have quantity to fill
     while (!far_side.empty() && remaining_quantity > 0) {
         const auto best_level = (side == Side::bid) ? far_side.begin()
             : std::prev(far_side.end());
         const auto& best_level_price = best_level->first;

         // 2. Stop matching if the best available price is no longer
             marketable
         if ((side == Side::bid && price < best_level_price) ||
             (side == Side::ask && price > best_level_price)) {
             break;
         }

         auto& best_level_orders = best_level->second;
         // 3. Execute against orders at the current best price level (
             Price-Time Priority)
         while (remaining_quantity > 0 && !best_level_orders.empty()) {
             auto& front_order = best_level_orders.front();

             if (const auto order_quantity = front_order.get_quantity();
                 remaining_quantity >= order_quantity) {
                 // 3a. Incoming order fully consumes the resting order
                 remaining_quantity -= order_quantity;
                 Trade new_trade = create_trade(order_id, front_order.
                     get_order_id(), broker_id, front_order.get_trader_id
                     (), ticker, front_order.get_price(), order_quantity,
                      side);
                 trade_publisher->try_publish(new_trade);
                 trade_events.emplace(std::move(new_trade));
                 order_id_map.erase(best_level_orders.front().
                     get_order_id());
                 best_level_orders.pop_front();
             } else {
                 // 3b. Resting order partially fills the incoming order
                 front_order.fill(remaining_quantity);
                 Trade new_trade = create_trade(order_id, front_order.
```

```
                    get_order_id(), broker_id, front_order.get_trader_id
                    (), ticker, front_order.get_price(),
                    remaining_quantity, side);
                trade_publisher->try_publish(new_trade);
                trade_events.emplace(std::move(new_trade));
                remaining_quantity = 0;
            }
        }

        // 4. Remove the price level if all resting orders are
            exhausted
        if (best_level_orders.empty()) {
            far_side.erase(best_level_price);
        }
    }

    // 5. If the incoming limit order is not fully filled, add it to
        the resting book (Maker)
    if (remaining_quantity > 0 && price != MARKET_BID_ORDER_PRICE &&
        price != MARKET_ASK_ORDER_PRICE) {
        order_id_map[order_id] = near_side[price].emplace(near_side[
            price].end(), order_id, price, remaining_quantity, side,
            broker_id);
    }
}
```

Listing 1: Limit Order Book Matching Algorithm

2.2.5 Distribution Layer: Market Data Processor

The Market Data Processor (MDP) is implemented as an independent C++ application running a continuous event loop. It instantiates dedicated shared-memory ring buffer consumers for each active trading symbol to process both trades and order book snapshots. During each iteration of its busy-wait loop, the MDP invokes non-blocking `try_pop()` operations on these queues to check for new events. When raw trade or order book data structures are detected, the processor utilizes the `nlohmann::json` library to serialize the binary C++ objects into JSON payloads. Finally, the serialized data is pushed asynchronously to all active subscribers via the WebSocket server, delivering tick-by-tick market updates to the frontend dashboard.

2.2.6 Simulation Layer: Agents & Market Regimes

The simulation layer was constructed utilizing a set of C++ autonomous agent classes (`NoiseTrader`, `MarketMaker`, and `InformedTrader`) that act as independent clients to the exchange. They interface with the system using the same `FixClient` SDK as human users. Agent actions are driven by background threads, with the `NoiseTrader` specifically utilizing standard library random number generators (`std::poisson_distribution` and `std::mt19937`) to model the stochastic arrival of retail orders. Meanwhile, the `InformedTrader` leverages a specialized `OracleService` process that simulates a fundamental asset price

track. The oracle disseminates this continuous price path by broadcasting updates over a dedicated WebSocket channel, which the informed agent consumes to continuously poll for price deviations against the live order book and execute its sniping strategy.

Listing 2 illustrates the core logic for the `MarketMaker` agent based on the Avellaneda-Stoikov model. The agent dynamically adjusts its reservation price and optimal spread based on real-time market variance and its current inventory risk.

```
void MarketMaker::make_market(const std::string& ticker) {
    OrderBookSnapshot ob = m_md_handler->get_latest_orderbook(ticker);
    double mid_price = ob.mid_price() == 0.0 ? 100.0 : ob.mid_price();
    double variance = m_md_handler->calculate_variance(ticker);
    variance = variance == 0.0 ? 0.01 : variance;

    double q = m_fix_client->get_inventory(ticker);
    double q_centered = q - m_initial_inventory;

    auto now = std::chrono::steady_clock::now();
    std::chrono::duration<double> elapsed = now - m_start_time;
    double time_left = std::max(0.0, (m_terminal_time - elapsed.count()
        ) / m_terminal_time);

    // Calculate Reservation Price (r)
    double reservation_price = mid_price - (q_centered * m_gamma *
        variance * time_left);

    // Calculate Optimal Spread
    double spread = (m_gamma * variance * time_left) + ((2.0 / m_gamma)
         * std::log(1.0 + (m_gamma / m_k)));

    double optimal_bid = reservation_price - (spread / 2.0);
    double optimal_ask = reservation_price + (spread / 2.0);

    place_quotes(ticker, optimal_bid, optimal_ask, spread);
}
```

Listing 2: Avellaneda-Stoikov Market Maker Strategy

2.2.7 Server Setup and Deployment

To orchestrate the system across a Linux environment, we developed a custom Python-based deployment script. This application functions as a localized control plane that parses TOML configuration files to manage the lifecycle of the various microservices. It automatically validates service names, resolves listening ports, and ensures services are fully ready or safely terminated. By handling the automated startup and teardown of the components, including the FIX Gateway, Order Manager, Matching Engine, and Market Data Processor, the deployment server provides a reliable and reproducible environment for running extensive market simulations.

2.2.8 Software Engineering Practices

Throughout the development of this complex system, we adhered to modern C++ software engineering principles and best practices to ensure robustness, maintainability, and performance:

- **Modern C++ Features:** We extensively utilized features from modern C++ standards (C++20/23) to write safer and more expressive code:
  - `std::expected`**:** Instead of relying heavily on exceptions, which can introduce non-deterministic control flow and performance overhead on the critical path, we used `std::expected` to return either a valid result or an explicit error state. This functional approach makes error handling predictable and type-safe.
  - **Monadic Operations (`and_then` / `transform` / `or_else`):** To access the values within a `std::expected` or `std::optional`, we utilized monadic chaining. Rather than writing repetitive `if-else` blocks to check for errors, monadic methods like `.and_then()` and `.transform()` allow us to specify what happens "after" a successful operation, while `.or_else()` defines the fallback behavior for errors. This avoids deep nesting of error-checking code and significantly improves readability across our network and service layers.
  - **Smart Pointers and RAII:** Resource Acquisition Is Initialization (RAII) and smart pointers (`std::unique_ptr`, `std::shared_ptr`) were systematically applied across all components. By utilizing the deterministic stack behavior of C++ classes, smart pointers automatically invoke their destructors when they go out of scope. This guaranteed deterministic resource management and strict prevention of memory leaks without relying on manual memory deallocation.
- **Design by Contract & Assertions:** To catch logic errors early, we adopted the "Design by Contract" methodology using the `boost::contract` library. We liberally applied preconditions, postconditions, and invariant assertions across critical components like the LOB and Balance Checker. This strict enforcement guarantees that functions only operate when their required state is valid, immediately asserting and failing fast if an invariant (such as an invalid order ID or negative balance) is violated.
- **Design Patterns & Abstractions:** We employed various structural and creational design patterns to decouple system components. The Factory Pattern (e.g., `MatchingEngineDependencyFactory`) was utilized to abstract the instantiation of complex dependencies in the Matching Engine and Order Manager. Furthermore, abstract interfaces (e.g., `InboundServer`, `Publisher`) and dependency injection

were used throughout the architecture. This allowed us to easily swap out implementations (for instance, substituting a real WebSocket server with a mock server during unit testing) without modifying the core business logic.

- **Asynchronous Logging (`spdlog`):** For observability, we integrated the high-performance `spdlog` library. To prevent I/O operations from blocking the main execution threads, we configured it with asynchronous sinks (`spdlog::async_factory`). The logger also supports configurable log levels via environment variables and includes comprehensive signal handlers (`SIGINT`, `SIGTERM`) to ensure all pending logs are safely flushed to disk before a graceful shutdown.

2.3 Testing

2.3.1 Test Plan and Methodology

Our testing strategy encompassed three primary phases: unit testing, integration testing, and performance benchmarking. We initially utilized the Catch2 library for our unit testing framework due to its lightweight and expressive nature. However, as the system's complexity grew, we migrated our test suite to Google Test (gtest). This transition was motivated by Google Test's robust mock-testing facilities (via Google Mock), which allowed us to effectively isolate components using dependency injection.

For unit testing, we verified the core components by simulating diverse order scenarios. A key focus was validating the Price-Time Priority algorithm to ensure orders were filled in the correct sequence. We also performed "death testing" using Google Test's `EXPECT_DEATH` macro (as seen in our `limit_order_book_test.cpp`) to verify that the system correctly terminates or asserts when encountering unrecoverable states, such as querying invalid order IDs.

Integration testing focused on the data flow from the Client SDK through the FIX Gateway and Order Manager to ensure that Protobuf serialization and WebSocket communication maintained data integrity. We tested the trade events emitted by the Matching Engine to ensure the end-to-end flow functioned correctly. Regarding stress testing, our high-load evaluations were scoped strictly to individual component benchmarking, such as isolating the Matching Engine's throughput and latency under massive order bursts.

A comprehensive execution matrix of the system's test suite, detailing critical scenarios and expected outcomes across multiple microservices, can be found in Appendix 6.5.

2.3.2 Locating Logic Errors and Bugs

Our testing procedures were critical in identifying flaws that were not initially apparent. A significant logic error was discovered regarding an LOB invariant violation. During

early testing, the Matching Engine incorrectly attempted to match against the worst ask price rather than the best ask price, as the engine was accessing the end of the ascending price list instead of the beginning. While simple unit tests missed this edge case, running a randomized order submission simulation revealed that "Ask" prices were crossing below "Bid" prices, which is a clear violation of market invariants.

This discovery prompted us to adopt the Design by Contract principle, writing more detailed test cases and enforcing preconditions, postconditions, and invariant assertions throughout the Matching Engine code to catch logic bugs early.

2.4 Evaluation

2.4.1 Benchmarking

We used Google Benchmark across multiple experimental configurations. All benchmarks were executed on an AMD Ryzen 5 6600H machine with 32 GB RAM running Linux, compiled with GCC utilizing `-O3` optimization flags. The baseline configuration is built in Release mode with assertions and contract checks disabled. The custom memory pool, vector-based LOB, and template specialization configurations are exploratory prototypes for potential future improvements, rather than immediate production defaults.

Matching Engine

Figure 6 compares the 10,000-order workload across the main latency-sensitive operations. The custom memory pool configuration is the only variant that consistently improves end-to-end performance over the baseline. Its strongest gain is in no-match order insertion (`BM_MatchingEngine_NewOrderNoMatchBurstLatency`), where latency is reduced from 0.807 ms to 0.434 ms ($\sim$1.86$\times$ faster). It also reduces pure LOB insertion latency from 0.672 ms to 0.257 ms ($\sim$2.61$\times$ faster). This significant speedup occurs because a custom memory pool pre-allocates contiguous blocks of memory at startup, completely bypassing the heavy operating system overhead of dynamic heap allocations (e.g., `malloc` or `new`) during rapid bursts of order insertions. In contrast, the template specialization configuration is mostly neutral to slightly slower, and the vector-based LOB configuration is effectively neutral for these engine-level benchmarks.

Legend for Figure 6:

- `lob-dsa`: Experiment replacing node-based data structures with contiguous `std::vector` layouts.
- `mem-pool`: Experiment utilizing a custom memory pool to pre-allocate order objects.

- `template`: Experiment utilizing C++ template specialization for execution paths.

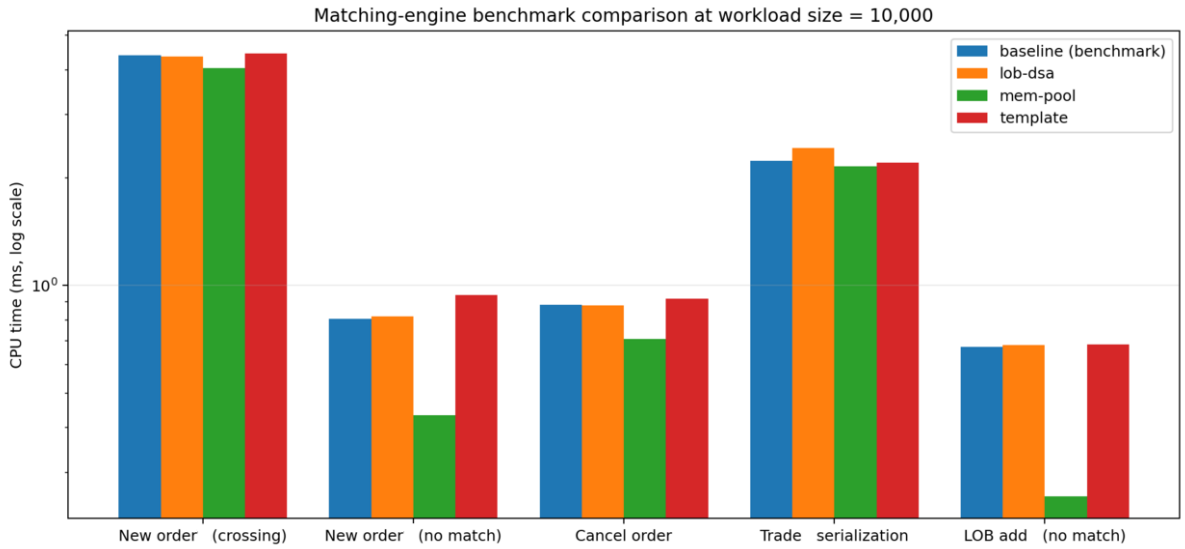

Figure 6: Matching-engine latency comparison across optimization branches at workload size 10,000 (log-scale y-axis).

To isolate the data-structure effect explored in the alternative data structure configurations, we also benchmarked a realistic mixed-flow LOB scenario up to 1,000,000 orders (Figure 7). The vector-based level layout improves large-scale throughput versus the map/list baseline (631.11 ms $\rightarrow$ 353.48 ms at 1,000,000 orders, $\sim$1.79$\times$ faster), while the flat-vector variant degrades sharply at scale due to expensive shifting and maintenance costs under heavy mixed updates. Specifically, inserting or removing an order in the middle of a massive, contiguous flat-vector requires shifting all subsequent elements, meaning the $\mathcal{O}(N)$ algorithmic cost eventually outweighs any hardware benefits gained from CPU cache locality. This result indicates that contiguous storage can outperform node-based structures when the access pattern remains locality-friendly, but only when insertion/cancellation costs are controlled.

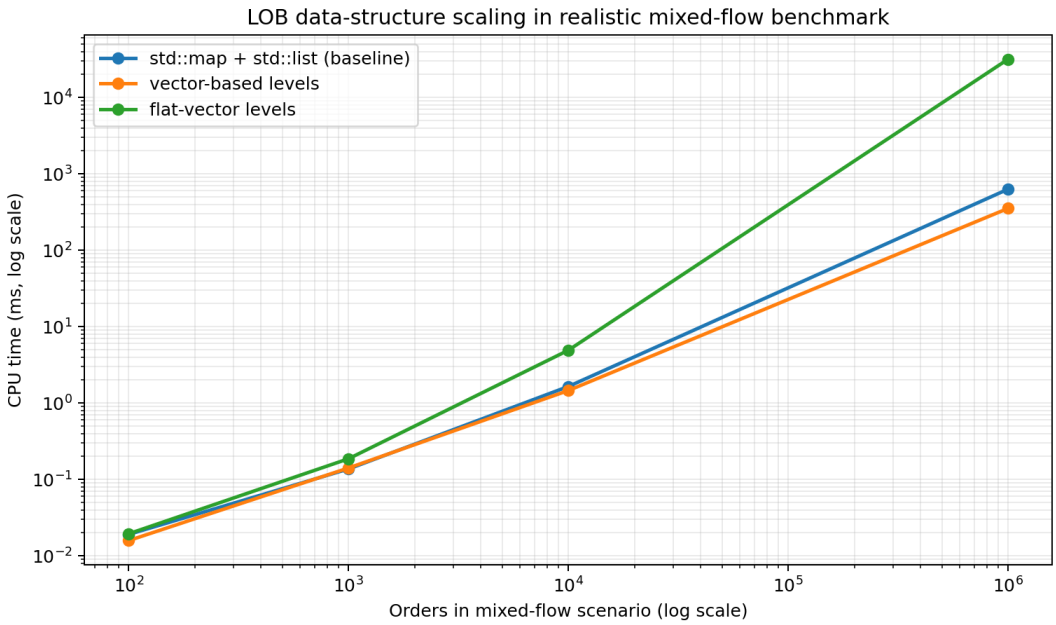

Figure 7: LOB data-structure scaling in realistic mixed-flow benchmarks testing alternative structures (log-scale axes).

Table 3 reports baseline matching-engine throughput for core 10,000-order workloads. Depending on operation type, throughput ranges from 2.28 million to 12.40 million operations per second.

Benchmark (10,000 workload)	**Latency (ms)**	**Throughput (ops/s)**
New order (crossing, with matching)	4.391	2,277,274
New order (no match)	0.807	12,395,537
Cancel order	0.882	11,333,297
Trade serialization	2.233	4,478,941

Table 3: Baseline matching-engine throughput in the default configuration.

These exploratory benchmarks successfully identified improvements to our initial design specification: moving from standard dynamic memory allocation to a custom memory pool is necessary for production-grade throughput, and node-based structures can be effectively replaced by contiguous arrays when carefully managing insertion and deletion costs.

While the average latency improvements are significant, our current profiling methodology heavily focuses on mean execution time and broad algorithmic speedups. High-frequency trading systems, however, are ultimately judged by their tail latency (e.g., 99th percentile) and performance stability under load. Future benchmarking should augment

these average-case results with a comprehensive jitter analysis, utilizing histograms to demonstrate that the system remains predictable during high message volume. Furthermore, to definitively validate our memory pool optimizations, subsequent profiling should incorporate hardware performance counters or kernel tracing tools. This granular breakdown would explicitly quantify reductions in CPU cache misses, branch mispredictions, and kernel-space context switching, proving whether the latency elimination is strictly due to cache locality or merely reduced allocation overhead.

Order Manager

We separately benchmarked Order Manager preprocessing and validation for the two request classes that follow different logic paths: new-order requests and cancel-order requests. Results show that cancel-order paths are consistently faster than new-order paths, both in preprocessing and validation, because they avoid symbol-side quantity checks and balance reservation updates performed for new orders.

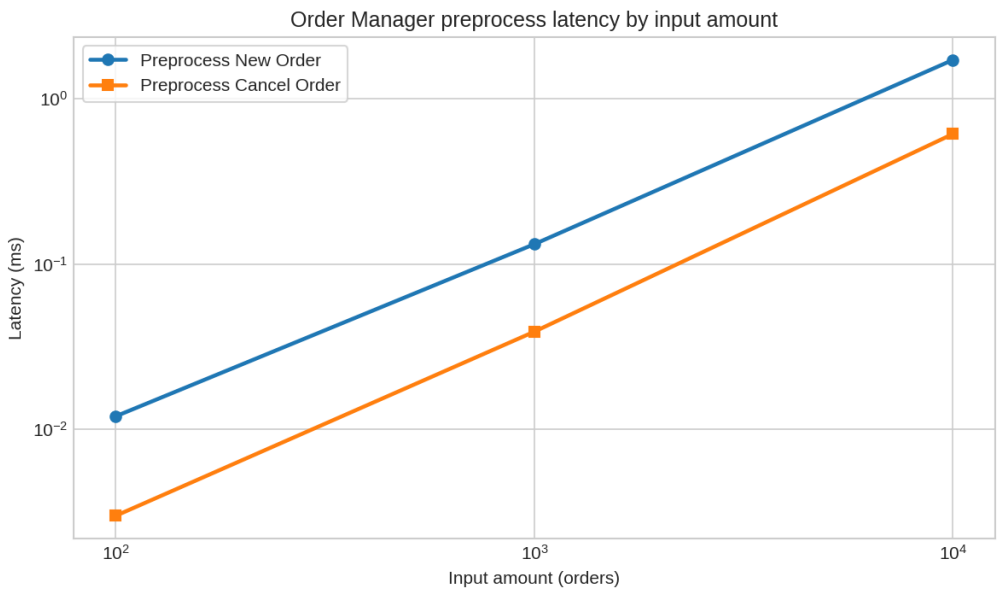

Figure 8: Order Manager preprocessing latency by input amount (new-order versus cancel-order paths; log-scale axes).

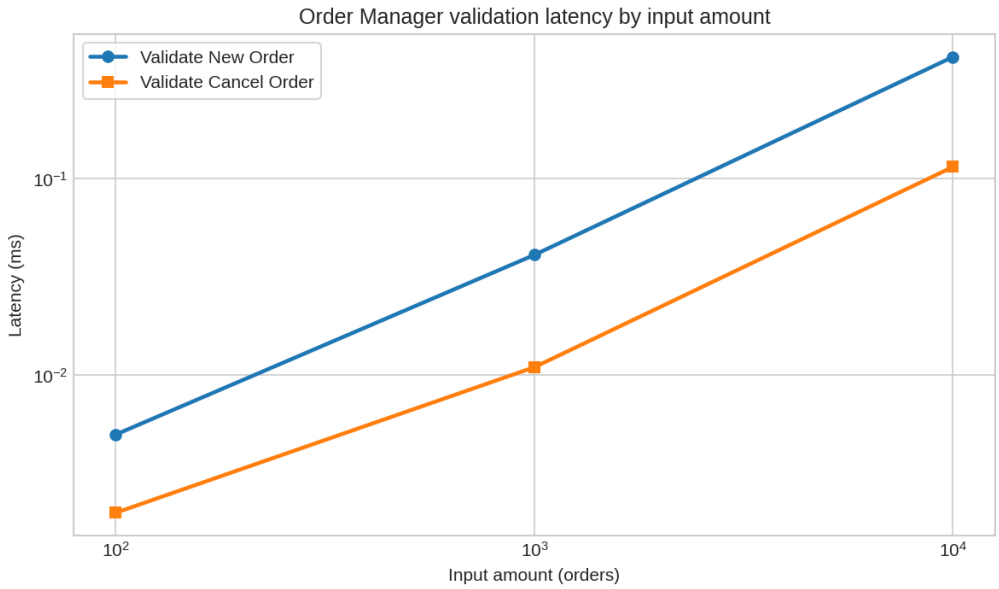

Figure 9: Order Manager validation latency by input amount (new-order versus cancel-order paths; log-scale axes).

At workload size 10,000, preprocessing latency is 1.72 ms for new orders and 0.613 ms for cancel requests. Validation latency is 0.417 ms for new orders and 0.115 ms for cancel requests. This corresponds to approximately 5.82M/16.30M ops/s for preprocessing and 23.96M/86.75M ops/s for validation (new/cancel respectively), confirming that path-specific benchmarking is necessary for accurate operation-level performance characterization.

Order Manager benchmark (10,000 workload)	**Latency (ms)**	**Throughput (ops/s)**
Preprocess new order	1.720	5,815,230
Preprocess cancel order	0.613	16,302,100
Validate new order	0.417	23,957,900
Validate cancel order	0.115	86,746,400

Table 4: Baseline Order Manager throughput on separated operation paths.

MPSC Lock-Free Ring Buffer

For the ring buffer component, we measured single-event producer-consumer handoff latency. In the baseline configuration rerun, we observed 771 ns in a single run, with aggregate mean 648.76 ns (manual-time) and 673 ns (CPU-time). This benchmark currently has one configured input amount (one event handoff), shown in Figure 10.

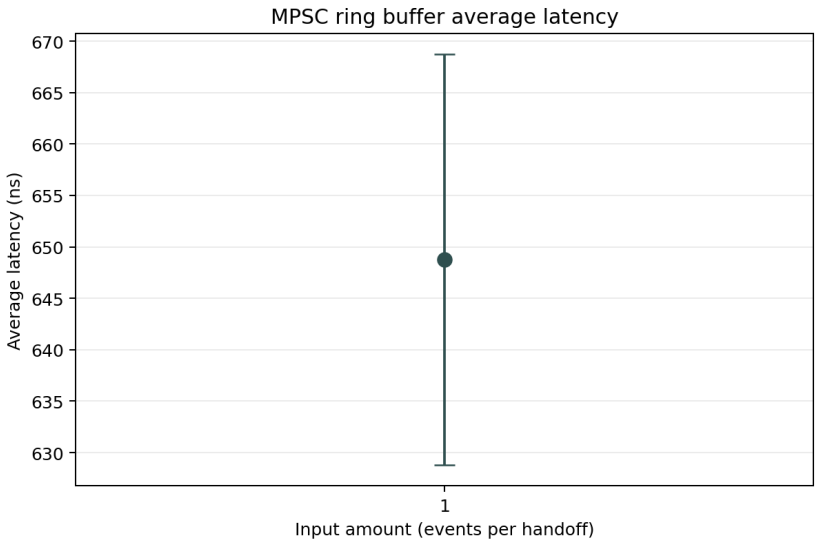

Figure 10: MPSC ring buffer average latency by configured input amount.

WebSocket

For the websocket benchmark on the baseline configuration, we measured mean round-trip latency at three payload sizes:

- 64 B: 0.03698 ms (∼27,042 round-trips/s)
- 512 B: 0.04106 ms (∼24,353 round-trips/s)
- 4096 B: 0.06227 ms (∼16,059 round-trips/s)

This shows clear latency growth as payload size increases in our local setup.

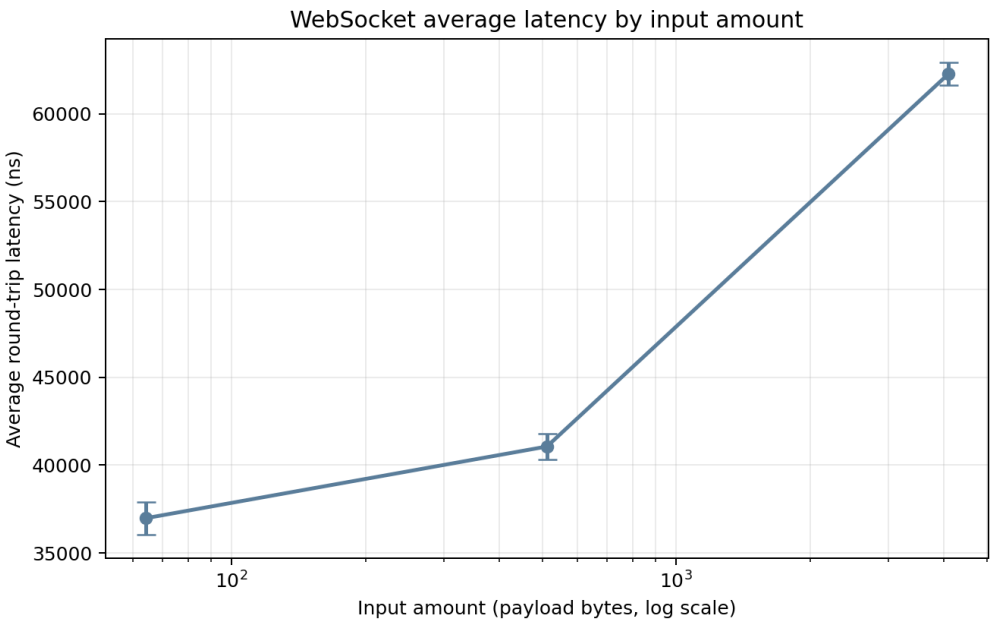

Figure 11: WebSocket average round-trip latency by input amount.

2.4.2 Simulation Realism

To evaluate the realism of the simulated market dynamics, we initialize the platform with a baseline configuration across the agent populations. Specifically, the simulation is initialized with 4 Market Makers, 3 Noise Traders, and 1 Informed Trader. The full, detailed list of default parameters for all agent types and the Oracle Service can be found in Appendix 6.6.

Using this baseline configuration, we analyze the resulting order book data to determine whether the simulation exhibits stylized facts commonly observed in real financial markets, guided by the aggregate criteria outlined in Jain et al.'s review of LOB simulations [3]. Specifically, we isolate the tick-by-tick market returns and bid-ask spread data for the `GME` ticker over a ∼48-minute highly volatile trading session (starting at $99.87 and peaking at $110.57).

Fat-Tailed Returns Real financial returns typically exhibit a leptokurtic (fat-tailed) distribution rather than a perfect Gaussian normal distribution, meaning extreme price movements occur more frequently than standard models predict. Our empirical analysis of the simulated `GME` trade returns (Figure 12) yielded a kurtosis of 5.5834 (where > 3 indicates fat tails). This confirms that the Oracle's jump-diffusion model, combined with informed trader sniping, accurately produces non-normal extreme price events.

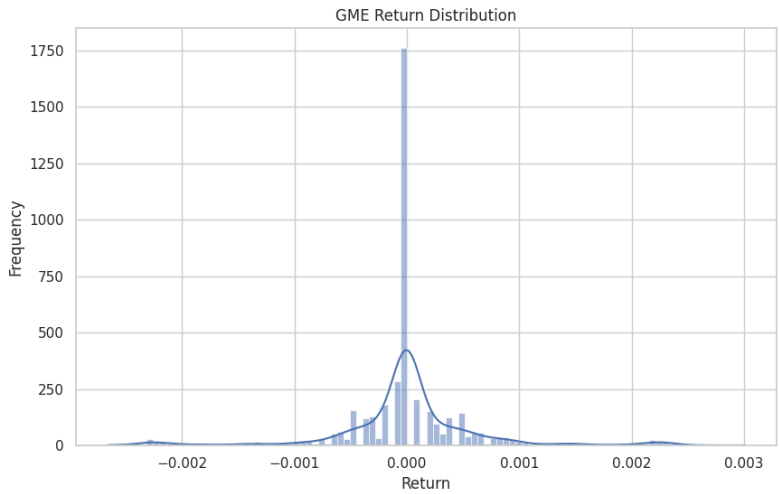

Figure 12: Distribution of `GME` trade returns showing fat tails.

Volatility Clustering Another universal stylized fact is volatility clustering: "large changes tend to be followed by large changes, and small changes by small changes." We measured this by plotting the Autocorrelation Function (ACF) of the absolute returns (Figure 13). The slowly decaying positive autocorrelation visually proves the presence of volatility clustering in the simulated market, accurately reflecting periods of high and low market stress.

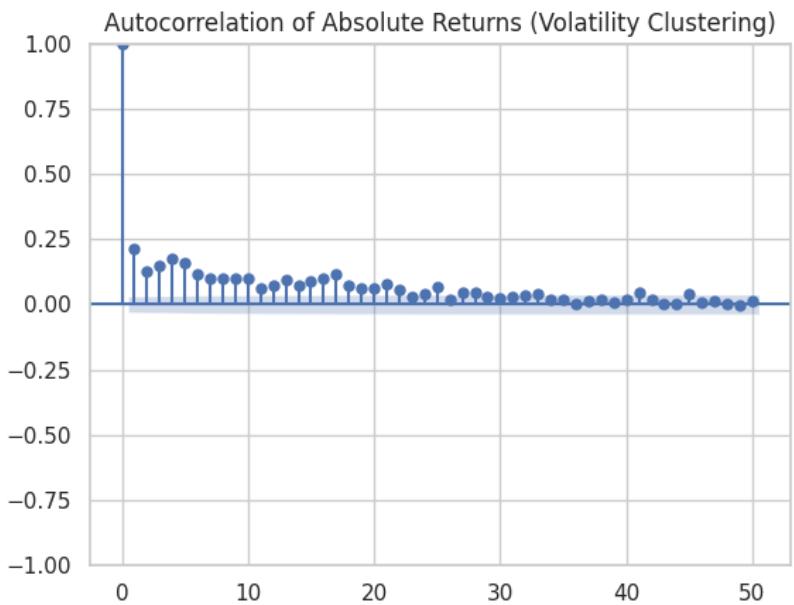

Figure 13: Autocorrelation of absolute returns demonstrating volatility clustering.

Bid-Ask Spread and Price Trajectory In a functioning LOB, active market makers should maintain a tight bid-ask spread despite volatility. Our simulation processed 232,442 total orders (115,262 bids, 117,180 asks) resulting in 4,340 executions. Figure 14 shows that the 4 Market Makers successfully competed to maintain a tight median spread of \$0.0006 (average \$0.0010), demonstrating robust liquidity provision. Simultaneously, Figure 15 illustrates the rapid intraday price discovery driven by the underlying fundamental jump-diffusion process.

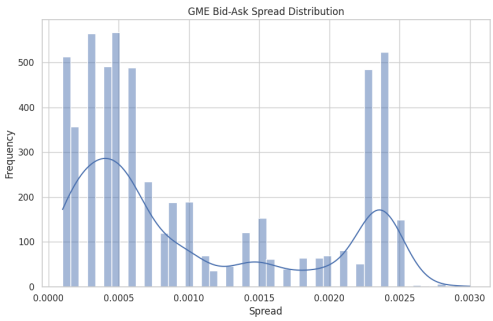

Figure 14: Bid-Ask spread distribution.

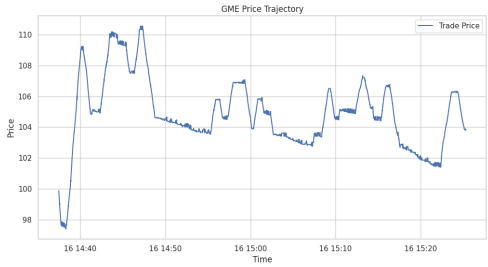

Figure 15: `GME` price trajectory.

Raw Return Autocorrelation As noted by empirical studies on market microstructure, price changes on extremely short time scales exhibit significant negative autocorrelation at the first lag, commonly attributed to the "bid-ask bounce" effect. Our simulation accurately reproduces this phenomenon (Figure 16), where the first lag shows sharp negative autocorrelation before mostly vanishing at further lags.

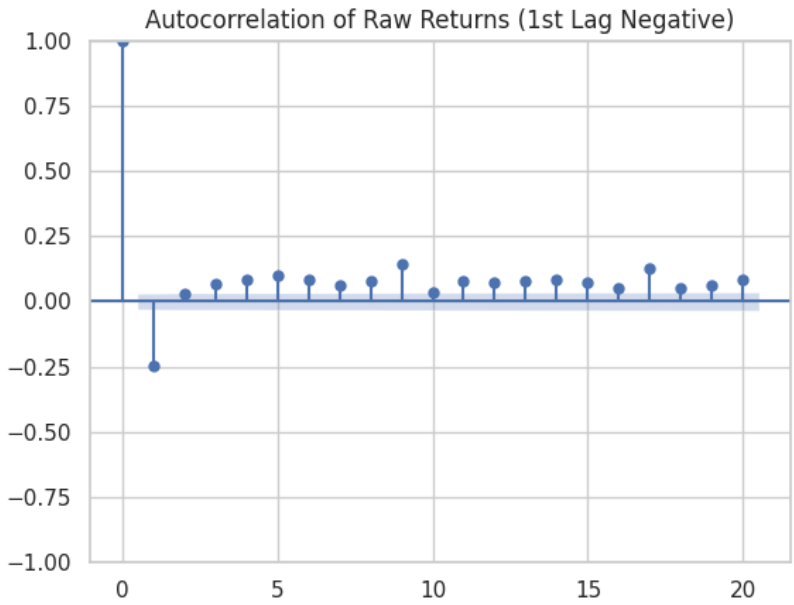

Figure 16: Autocorrelation of raw returns showing negative first lag (bid-ask bounce).

Average Shape of the Order Book A key empirical ground truth for simulator fitness is the average shape of the order book. Figure 17 presents the mean volume at each price level relative to the mid-price. The simulation exhibits a clear inverted 'V' shape typical of low-spread, highly liquid assets, demonstrating that our Avellaneda-Stoikov market making agents maintain realistic volume distributions across depth levels.

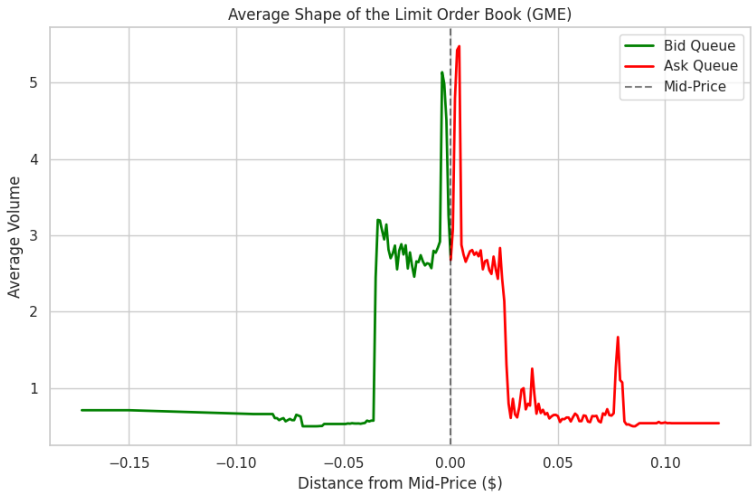

Figure 17: Average shape of the LOB relative to mid-price.

3 Discussion

This project successfully delivered a highly modular and functional simulated exchange environment, but it also served as a profound learning experience regarding the extreme engineering constraints of algorithmic trading systems. In this section, we evaluate the deeper reasons behind our successes and shortcomings, and discuss the architectural limitations that would prevent this system from being deployed in a live, high-frequency trading (HFT) production environment.

3.1 Project Evaluation and Reflections

We are most confident in our achievement of building a low-latency, strictly correct LOB and an intuitive C++ SDK. Our success here stems directly from our rigorous adoption of modern C++ paradigms, strict memory management (RAII), and our choice of lock-free data structures. By explicitly isolating the matching engine from network and database I/O, we ensured that the core matching logic which is the most critical part of the system, could perform optimally without being bottlenecked by external dependencies.

Our approach to the system's distributed architecture also went exceptionally well in terms of logical decoupling. Separating concerns across the FIX Gateway, Order Manager, Matching Engine, and Market Data Processor allowed us to build an elegant "game server" provisioning model.

Regarding market simulation, we successfully replicated key microstructural stylized facts through our implementation of the Avellaneda-Stoikov market maker and fundamental jump-diffusion models, such as fat-tailed returns, volatility clustering, tight bid-ask spreads, and the bid-ask bounce. However, it should be noted that these phenomena were achieved primarily through trial-and-error tuning of model parameters. Because we have not yet performed a rigorous sensitivity analysis, we cannot definitively claim the platform is a fully robust high-frequency trading testbed. Furthermore, generating truly realistic macroscopic market behaviour over extended periods requires vastly more sophisticated parameter calibration and multi-agent interaction than a single project timeframe permits. While microstructural dynamics were accurately captured under specific configurations, perfectly replicating long-term real-world trends would require deeper mathematical modelling of human psychological trading factors and macro-economic events.

3.2 System Limitations

While our platform serves perfectly as an educational sandbox, several architectural shortcuts prevent its use in a real-world financial institution:

- **Network Latency and IPC Overhead:** We relied heavily on WebSockets and JSON serialization for inter-service communication. This encapsulation overhead and computational expense prevents nanosecond-scale determinism.
- **Deployment and Robustness:** The system currently utilizes a local Python script for orchestration and lacks auto-recovery mechanisms. Consequently, any microservice crash results in dropped messages and creates single points of failure.
- **Concurrency and Scalability:** Our Matching Engine currently processes orders sequentially. Furthermore, relying on a single-node QuestDB instance for tick data storage will eventually bottleneck under heavy concurrent load.
- **Flow Control:** The system lacks strict rate-limiting at the gateway layer, leaving the platform vulnerable to buffer overflows and memory exhaustion during sudden bursts of massive order flow.

4 Conclusions and Future Work

This project successfully culminated in the development of a highly modular, low-latency simulated stock exchange designed for algorithmic trading education. Our major technical accomplishments include the implementation of a mathematically rigorous LOB, an intuitive C++ Trading SDK, a dual-database persistence layer, and a modern React-based frontend that orchestrates isolated trading "game servers." Throughout this process, we gained profound insights into the stringent performance and reliability constraints of financial microservices, particularly regarding modern C++ memory management, IPC shared memory, and event-driven architectures.

To resolve the system limitations identified and evolve this platform into a production-grade algorithmic trading infrastructure, we recommend the following avenues for future work:

- Future iterations should replace WebSockets and JSON for internal communication with zero-copy binary protocols (e.g., FlatBuffers or Simple Binary Encoding) and ultra-low-latency transport alternatives. These include Unix domain pipes for co-located services, UDP multicast, or hardware-accelerated kernel bypassing techniques (e.g., `AF_XDP` or DPDK) to eliminate operating system networking stack overhead.
- Transitioning from local Python orchestration to true CI/CD pipelines deploying across distributed AWS EC2 instances. This should be paired with hardware load balancers and robust auto-recovery mechanisms to ensure seamless failover, such as hot-standby replicas managed via distributed consensus and persistent event sourcing.
- The Matching Engine's throughput can be drastically improved by assigning independent CPU threads to different tickers, yielding near-linear scaling without lock contention (e.g., utilizing the LMAX Disruptor pattern). Additionally, database sharding should be implemented to partition user and tick data across multiple nodes.
- Incorporating more sophisticated mathematical models or AI-driven agents could significantly improve macroscopic market realism. Furthermore, conducting extensive user testing with volunteer algorithmic traders would provide invaluable data on the SDK's usability and evaluate platform stability under unpredictable, malicious, or massive loads (e.g., implementing and rigorously testing rate-limiting at the gateway layer).

5 References

References

[1] M. Aquilina, E. Budish, and P. O'Neill, "Quantifying the High-Frequency Trading 'Arms Race': A New Methodology and Estimates," Financial Conduct Authority, Occasional Paper no. 50, 2020. [Online]. Available: `https://www.fca.org.uk/publication/occasional-papers/occasional-paper-50.pdf`. [Accessed: Oct. 1, 2025].

[2] M. Avellaneda and S. Stoikov, "High-frequency trading in a limit order book," *Quantitative Finance*, vol. 8, no. 3, pp. 217–224, 2008.

[3] K. Jain, N. Firoozye, J. Kochems, and P. Treleaven, "Limit Order Book Simulations: A Review," *arXiv preprint arXiv:2402.17359*, Feb. 2024.

[4] B. Nigito, "How to Build an Exchange," Jane Street Tech Talks, Jane Street Group, LLC, 2025. [Online]. Available: `https://www.janestreet.com/tech-talks/building-an-exchange/`. [Accessed: Oct. 1, 2025].

[5] Quod Financial, "Open Source Multi-Asset Market Simulator: Test Trading Strategies - Quant Replay," QuantReplay, 2025. [Online]. Available: `https://quantreplay.com/`. [Accessed: Oct. 1, 2025].

[6] wkselph, "How to Build a Fast Limit Order Book," *WK's High Frequency Trading Blog*, Feb. 15, 2011. [Online]. Available: `https://web.archive.org/web/20110219163448/http://howtohft.wordpress.com/2011/02/15/how-to-build-a-fast-limit-order-book/`. [Accessed: Oct. 1, 2025].

6 Appendix

6.1 Minutes of Project Meeting

6.1.1 Minutes of the 1st Project Meeting

Date: May 5, 2025
Time: 2:00 pm
Place: Room 3504
Present: Gordon, Josh, Om, Prof. Sander
Absent: None
Recorder: Josh

1. Approval of minutes
   There were no minutes to approve since this was the first formal group meeting.

2. Report on progress
   (a) All members have read and understood the Final Year Project instructions.

   (b) All members have researched several potential FinTech-related topics.

3. Discussion items
   (a) Om introduced three topics that we were interested in working on, including market simulation for educational algorithmic trading, design and implementation of an exchange architecture, and simulation of a LOB for high-frequency trading. Om asked whether Prof. Sander was willing to supervise our Final Year Project.

   (b) Gordon pointed out that the first and the third topics were similar, and we might consider merging them into a new topic, for example, the implementation of an algorithmic trading platform that provided a simulated market environment with a highly efficient LOB design.

   (c) Prof. Sander showed interest in the topics mentioned. He pointed out that the focus of these topics was diverse, from benchmarking the efficiency of different data structures for the LOB to system design. He expressed his willingness to supervise our project and suggested that we write a draft proposal with details to provide more context before narrowing down the direction in which we wanted to move forward.

4. Goals for the coming month
   All members will do further research on the three topics closely.
   A draft proposal will be sent to Prof. Sander by May 11, 2025.

5. Meeting adjournment and next meeting
   The meeting was adjourned at 2:45 pm.
   The next meeting will be at 2:00 pm on May 12th at Room 3504.

6.1.2 Minutes of the 2nd Project Meeting

Date: May 12, 2025
Time: 2:00 pm
Place: Room 3504
Present: Josh, Om, Prof. Sander
Absent: Gordon
Recorder: Josh

1. Approval of minutes
   The minutes of the last meeting were approved without amendment.

2. Report on progress
   (a) All members have done further analysis on the proposed topics during the last meeting.

   (b) Om drafted the proposal explaining each topic in detail, which was sent to Prof. Sander.

3. Discussion items
   (a) Prof. Sander said that he read the proposal and found it interesting. He suggested that we spend the coming summer on research because there was time remaining before we submitted the final proposal. He also advised us to set up a Zoom channel for easier communication.

   (b) Om agreed with Prof. Sander's advice.

4. Goals for the coming three months
   All members will do a deeper analysis and write up a proposal with the finalised topic.

5. Meeting adjournment

The meeting was adjourned at 2:30 pm.

6.1.3 Minutes of the 3rd Project Meeting

Date: Aug 21, 2025
Time: 8:30 am
Place: Economy Class of Flight UO612, from Hong Kong to Fukuoka, Japan
Present: Gordon, Om
Absent: Josh
Recorder: Gordon

1. Approval of minutes
   The minutes of the last meeting were approved without amendment.

2. Report on progress
   (a) All members narrowed down the final topic to market simulation for algorithmic trading, with high efficiency implementation of some core components of an exchange architecture.

   (b) All members have started working on the final proposal.

3. Discussion items
   (a) Gordon pointed out that the coverage of our finalised topic was a bit large, so it made more sense to divide the development plan into different phases.

   (b) Om suggested three phases. At phase 1, the core market mechanism is implemented, including the LOB, trade interface, and historical data handler, enabling users to host locally and backtest their own strategy with static historical market data. At phase 2, the system evolves into a centralized server, and the users submit orders by communicating with the server via HTTP or WebSocket requests. At phase 3, we implement a model that can simulate behaviours of different parties in the market, including market makers, institutions, individual investors, etc, to make the market environment more realistic.

   (c) Gordon estimated the man-hours required for each task.

4. Goals for the coming week
   All members will do competitor analysis, as well as research on existing exchange architectures.

5. Meeting adjournment
   The meeting was adjourned at 9:30 am.

6.1.4 Minutes of the 4th Project Meeting

Date: Aug 23, 2025
Time: 8:00 pm
Place: Royal Park Canvas, Fukuoka, Japan
Present: Josh, Gordon, Om
Absent: None
Recorder: Josh

1. Approval of minutes
   The minutes of the last meeting were approved without amendment.

2. Report on progress
   (a) Gordon has done a competitor analysis.

   (b) Josh and Om have researched existing exchange architectures.

3. Discussion items
   (a) Gordon shared his findings. He mentioned that many crypto exchanges, like Binance and OKX, provide a set of trading API for simulation only. Apart from that, IMC holds an annual trading competition named IMC Prosperity. The participants are given a Python SDK to trade with a set of pre-defined market bots and try to earn as much as possible.

   (b) Josh said that with their market data and exchange infrastructure, these exchanges can provide very similar services and questioned how we can have an edge against them.

   (c) Om shared the research results of some current exchange architectures. He pointed out that the LOB is one of the most important core components, which we should put much effort into developing for the best performance. Furthermore, a typical exchange architecture also includes other components, including the exchange gateway, sequencers, market data processor, and so on. He said it was not feasible to implement all components within the scope of this project, and we might need to choose between them based on our use cases.

4. Goals for the coming week

(a) All members will continue working on the proposal report.

(b) All members will study closely to narrow down the required components for the project.

5. Meeting adjournment
   The meeting was adjourned at 9:30 pm.

6.1.5 Minutes of the 5th Project Meeting

Date: Sep 19, 2025
Time: 3:00 pm
Place: Room 4368
Present: Josh, Gordon
Absent: Om
Recorder: Josh

1. Approval of minutes
   The minutes of the last meeting were approved without amendment.

2. Report on progress
   (a) All members have worked together on finishing the proposal report.

   (b) All members have started learning advanced C++ techniques.

3. Discussion items
   (a) Gordon suggested we start designing the architecture of the local backtester for phase 1 of the project. He proposed a client-server architecture where the users interact with the matching engine via API because that is what we can reuse for phase 2.

   (b) Josh said that with the client-server design the user is not guaranteed to get the same backtest result every time because of asynchronization, which was counter intuitive as a backtester.

   (c) Gordon agreed with Josh's point, and suggested a mechanism to synchronize the client with the server which only applies to backtester only.

   (d) Josh expressed his concern about the overhead by doing so. But he agreed that it is important to produce reusable components and interfaces for later

phases. He then refined the architecture diagram.

4. Goals for the coming week
   (a) All members will continue learning C++.

   (b) All members will start designing the components they are responsible for developing.

5. Meeting adjournment
   The meeting was adjourned at 5:00 pm.

6.1.6 Minutes of the 6th Project Meeting

Date: Oct 1, 2025
Time: 10:00 am
Place: Room 4368
Present: Josh, Gordon, Om
Absent: None
Recorder: Om

1. Approval of minutes
   The minutes of the last meeting were approved without amendment.

2. Report on progress
   (a) All members have designed their components.

   (b) All members have started learning advanced C++ techniques.

3. Discussion items
   (a) Gordon shared his design of the matching engine. He suggested each matching engine should hold multiple LOBs, each of which is a node-based data structure for O(log n) time complexity for all major operations like search, insertion and deletion.

   (b) Om referred to a research paper and argued that low time complexity does not imply high performance in reality. Using std::vector is more cache-friendly and performance boosting.

   (c) Josh shared his design of the SDK for clients to interact with our system. Om shared his design of the gateway.

(d) Josh and Om discussed the protocol and parameters used for the communication between the client SDK and the gateway. FIX4.2 was agreed to be used.

4. Goals for the coming week
   (a) All members will start developing the components.

   (b) Gordon will benchmark the performance difference between a node-based data structure and a linear std::vector for the matching engine.

5. Meeting adjournment
   The meeting was adjourned at 1:00 pm.

6.1.7 Minutes of the 7th Project Meeting

Date: Oct 28, 2025
Time: 4:00 pm
Place: Room 4368
Present: Josh, Gordon, Om
Absent: None
Recorder: Gordon

1. Approval of minutes
   The minutes of the last meeting were approved without amendment.

2. Report on progress
   (a) Gordon has benchmarked the performance difference between different data structures for the matching engine.

   (b) All members have completed the first draft of their components, including client SDK, gateway and matching engine.

   (c) Client SDK and gateway have been demonstrated to be integrated well together.

3. Discussion items
   (a) Gordon shared his benchmark results, which showed that std::vector was indeed better in terms of operation latency.

   (b) Gordon suggested that we backlog replacing node-based data structure with

std::vector for the matching engine since the priority should be completing the implementation and integration test. Josh and Om agreed with that.

(c) Josh shared the integration test result. Order requests can be successfully sent from client SDK to the gateway.

(d) Om suggested adding a new component named order manager to our architecture, and removing the historical data handler and the backtester, since it is no longer one of the focuses. Josh and Gordon agreed.

4. Goals for the coming week
   (a) Josh will start implementing the market data processor component.

   (b) Gordon will start working on the order manager.

   (c) Om will start building our internal websocket client and server for different service use.

5. Meeting adjournment
   The meeting was adjourned at 4:45 pm.

6.1.8 Minutes of the 8th Project Meeting

Date: Nov 15, 2025
Time: 11:00 am
Place: Room 4368
Present: Josh, Gordon, Om
Absent: None
Recorder: Josh

1. Approval of minutes
   The minutes of the last meeting were approved without amendment.

2. Report on progress
   (a) Gordon has implemented the handler of order manager which handles all incoming order requests from gateways.

   (b) Josh has implemented a multiple-producer-single-comsumer (MPSC) lock-free ring buffer backed by shared memory, for fast data sharing between processes.

(c) Om has integrated the third-party library websocketpp into the project for our internal websocket client and server use.

3. Discussion items
   (a) Josh shared his design choice for the market data processor using the MPSC lock-free ring buffer.

   (b) Gordon questioned if the ring buffer really helped with performance improvement.

   (c) Om recalled the difference between lock-free and wait-free, and suggested we verify if the ring buffer is actually lock-free. Josh agreed with that.

   (d) Josh pointed out that clearer error messages should be provided by our websocket library.

4. Goals for the coming week
   (a) Josh will continue the development of the market data processor.

   (b) Gordon will continue implementing the order manager.

   (c) Om will verify the performance improvement of the ring buffer and if it is truly lock-free.

5. Meeting adjournment
   The meeting was adjourned at 12:30 pm.

6.1.9 Minutes of the 9th Project Meeting

Date: Dec 5, 2025
Time: 4:30 pm
Place: Room 3504
Present: Gordon, Om, Prof. Sander
Absent: Josh
Recorder: Gordon

1. Approval of minutes
   The minutes of the last meeting were approved without amendment.

2. Report on progress

Om reported the current progress to Prof. Sander, including the client SDK, gateways, order manager and market data processor.

3. Discussion items
   (a) Gordon demonstrated that an order request sent by the client SDK can be received and recognised correctly by the destination gateway.

   (b) Prof. Sander thought the work was great.

4. Goals for the coming week
   (a) All members will continue the development of their components.

   (b) Josh will initiate the development of a simple frontend to visualize the LOB and trade data.

5. Meeting adjournment
   The meeting was adjourned at 4:45 pm.

6.1.10 Minutes of the 10th Project Meeting

Date: Jan 10, 2026
Time: 3:00 pm
Place: Room 4368
Present: Josh, Gordon, Om
Absent: None
Recorder: Josh

1. Approval of minutes
   The minutes of the last meeting were approved without amendment.

2. Report on progress
   (a) Josh finished the implementation of frontend using React and TypeScript. Besides, the market data processor has been completed.

   (b) Om refined the internal websocket library to have better error messages for easier debugging. The library now can handle multiple connections.

   (c) Gordon has finished the first draft of order manager.

3. Discussion items

(a) Josh demonstrated that data from the matching engine can now be visualised on the frontend dashboard.

(b) All members agreed that integration tests should be conveyed to ensure all services can work well together.

4. Goals for the coming week
   (a) Gordon will develop the balance checking feature for order manager.

   (b) All members will work on integration tests.

5. Meeting adjournment
   The meeting was adjourned at 5:00 pm.

6.1.11 Minutes of the 11th Project Meeting

Date: Feb 5, 2026
Time: 3:30 pm
Place: Room 3504
Present: Josh, Gordon, Om, Prof. Sander
Absent: None
Recorder: Om

1. Approval of minutes
   The minutes of the last meeting were approved without amendment.

2. Report on progress
   (a) Josh has implemented a generator to generate mock market data to the matching engine. The events happening in the matching engine will be sent to the market data processor via shared memory, then forwarded to clients and frontend dashboard.

   (b) Om has enhanced the gateway implementation. Now the system can support multiple gateways to receive requests from clients, and load balancing has been implemented.

   (c) Gordon has finished the integration between gateway and order manager.

   (d) The team is now working on integrating all components and making sure the data flows correctly and seamlessly.

3. Discussion items
   (a) Gordon explained some technical challenges faced during the past few weeks, including CMake compilation errors and low latency tactics.

   (b) Prof. Sander suggests that we implement our own memory allocator to optimize the standard library performance for our use case.

   (c) Josh demonstrated the data flow in the system with the frontend dashboard.

4. Goals for the coming week
   (a) All members will continue work on integration tests.

   (b) All members will benchmark and optimize the performance of the components of which they are in charge.

5. Meeting adjournment
   The meeting was adjourned at 3:50 pm.

6.1.12 Minutes of the 12th Project Meeting

Date: Feb 24, 2026
Time: 3:00 pm
Place: Room 4368
Present: Josh, Om, Gordon
Absent: None
Recorder: Josh

1. Approval of minutes
   The minutes of the last meeting were approved without amendment.

2. Report on progress
   (a) Josh completed a first draft of the deployment flow for creating a new server instance and onboarding all required services with environment-specific configuration.

   (b) Om and Gordon reviewed how deployment settings are validated before service startup and confirmed the required runtime parameters for gateway and backend services.

3. Discussion items
   (a) The team discussed setting up a deployment pipeline to standardize server creation, including config validation, service bootstrapping, and post-deployment health checks.

   (b) Josh proposed packaging the deployment steps into a repeatable pipeline so that a new server can be provisioned and started with minimal manual edits.

   (c) Gordon suggested adding explicit rollback and restart procedures in case one service fails during bring-up. Om agreed and proposed documenting the operational runbook alongside the pipeline scripts.

4. Goals for the coming week
   (a) Josh will implement and refine the deployment pipeline for creating a new server.

   (b) All members will review the deployment checklist and validate it against the current service topology.

5. Meeting adjournment
   The meeting was adjourned at 4:00 pm.

6.1.13 Minutes of the 13th Project Meeting

Date: Mar 3, 2026
Time: 3:00 pm
Place: Room 4368
Present: Josh, Om, Gordon
Absent: None
Recorder: Gordon

1. Approval of minutes
   The minutes of the last meeting were approved without amendment.

2. Report on progress
   (a) Gordon reviewed the current order manager flow and identified the remaining work for stricter balance checking on incoming orders.

   (b) Josh and Om validated that order lifecycle events are already available to support persistence updates once the database writes are finalized.

3. Discussion items
   (a) The team discussed order manager refinement focused on two priorities: preventing insufficient-balance executions and persisting balance updates to the database consistently after trades.

   (b) Gordon proposed tightening validation so insufficient balance is rejected before forwarding orders to the matching engine, and ensuring balance deltas are applied atomically with database persistence.

   (c) Josh suggested extending integration tests to cover insufficient-balance rejection and successful persistence paths. Om agreed and proposed adding logs around balance update points to simplify debugging.

4. Goals for the coming week
   (a) Gordon will complete the order manager refinement, including balance checking and persistence to the database.

   (b) All members will run end-to-end checks to verify balances remain consistent across services after execution.

5. Meeting adjournment
   The meeting was adjourned at 3:50 pm.

6.1.14 Minutes of the 14th Project Meeting

Date: Mar 25, 2026
Time: 2:30 pm
Place: Room 4368
Present: Josh, Om, Gordon
Absent: None
Recorder: Om

1. Approval of minutes
   The minutes of the last meeting were approved without amendment.

2. Report on progress
   (a) Om reviewed the simulation deployment modules for market makers, noise traders, and informed traders, and mapped how each bot type is started and configured.

(b) Josh and Gordon confirmed the expected bot-account naming and startup sequence needed for repeatable simulation runs.

3. Discussion items
   (a) The team discussed implementing simulation bots to improve market realism by running coordinated market maker, noise trader, and informed trader agents in the same session.

   (b) Om proposed a staged rollout: first stabilize deployment and lifecycle control of each bot type, then tune parameters and inventories for realistic activity patterns.

   (c) Gordon suggested defining baseline scenarios and metrics (spread, fill rate, and trade intensity) so bot behavior can be evaluated quantitatively. Josh agreed and proposed wiring these scenarios into the existing simulation configuration pipeline.

4. Goals for the coming week
   (a) Om will implement and integrate simulation bots across the deployment workflow.

   (b) All members will calibrate default parameters and verify that bot interactions produce stable market dynamics.

5. Meeting adjournment
   The meeting was adjourned at 3:20 pm.

6.2 Explanation on Limit Order Books

Electronic markets are governed by centralized Limit Order Books (LOBs), where heterogeneous agents submit, cancel, and execute orders that collectively form price, liquidity, and volatility dynamics. Each LOB contains a buy side (also called bids) and a sell side (also called asks). If the bid price is greater than or equal to the ask price, the quantity matched will be removed from the LOB. Orders are first sorted by price, then sorted by the earliest time of arrival. This is known as *price-time priority*.

For example, suppose we start with an empty LOB. A trader submits a buy order for one share of APPL at $100. Since there is no one selling the desired share, this order sits at the "top" of the LOB as the best bid. When another trader submits a buy order

of one share of APPL at $99, this bid sits *behind* the $100 bid due to price-time priority. If another trader then submits a sell order for one APPL share at $100, there is now a *match* between bid and ask prices, and those orders are filled. All that remains on the LOB is the bid at $99, becoming the new best bid. If a seller places an ask for $105, it is clear that no matching is available. This is known as a *bid-ask spread* of $5. The above example describes limit orders. Another popular type of orders are market orders, which match with the best prices immediately.

This simple mechanism allows for market participants to provide and extract information regarding an asset. For example, a larger bid-ask spread is correlated with lower liquidity, reflecting a riskier trade when entering a position. By aggregating buyer and seller intentions in a central LOB, traders are able to infer the fair price of an asset.

6.3 Project Planning

6.3.1 Distribution of Work & Gantt Chart

G - Gordon J - Josh O - Om										
Task	**PIC**	**Aug**	**Sep**	**Oct**	**Nov**	**Dec**	**Jan**	**Feb**	**Mar**	**Apr**
Perform Literature Survey	O									
Write Proposal Report	G									
Design & Implement LOB	J									
Design & Implement Gateway	O									
Design & Implement Order Manager	G									
Design & Implement Market Data Processor	J									
Design & Implement Trade Interface and SDK	J									
Test LOB	G									
Test Gateway	O									
Test Order Manager	G									
Test Market Data Processor	J									
Test Trade Interface and SDK	J									
Design & Implement System Architecture & Infrastructure	G									
Design & Implement Traffic & Connection Management	O									
Test System Architecture & Infrastructure	G									
Test Traffic & Connection Management	O									
Design, Implement & Test Market Simulation Models	O									
Write Progress Report	J									
Write Final Report	G									
Record Presentation Video	J									
Prepare Presentation Deck	O									

Table 5: Project Timeline and Task Distribution

6.4 Required hardware & software

6.4.1 Hardware

- Linux x86_64 machine (Fedora 43, kernel 6.19.8)
- CPU: AMD Ryzen 5 6600H (6 cores / 12 threads, up to 4.57 GHz)
- Memory: 32 GiB RAM

6.4.2 Software

- React and JavaScript for frontend development
- C++23 as core programming language
- PostgreSQL (locally hosted)
- QuestDB (locally hosted)

6.5 Comprehensive Test Cases

Table 6 provides a comprehensive matrix of test cases extracted from the Google Test suites across multiple backend services (Matching Engine, Order Manager, and Backend Service) and Catch2 suites for common libraries. While we initially used Catch2 for its lightweight nature and continued to use it for our base libraries, we migrated to Google Test for our complex backend services to leverage its robust mocking framework. This matrix serves as evidence of the rigorous execution testing and validation of system invariants.

Table 6: Comprehensive execution matrix of test cases across backend services and common libraries.

Test Name	**Component**	**Scenario/ Description**	**Expected Result**	**Actual Result**	**Pass /Fail**
`MatchMultiple StandingBid Orders`	Matching Engine	Aggressive ask order matches against multiple limit buy orders at the same price level.	Orders matched strictly in arrival time sequence (Price-Time Priority).	Matches generated in correct temporal order.	Pass

Continued on next page

Table 6: Comprehensive execution matrix of test cases across backend services and common libraries (continued)

Test Name	**Component**	**Scenario/ Description**	**Expected Result**	**Actual Result**	**Pass /Fail**
`CompletelyFill StandingBid Order`	Matching Engine	Aggressive ask order crosses and exactly fills resting bid order.	Single trade executed; resting bid completely removed from LOB.	Trade generated; LOB updated correctly.	Pass
`PartiallyFill MarketAsk`	Matching Engine	Market ask order partially filled by available resting bids.	Trade generated for available quantity; market order fully consumed.	Trade generated; remainder of book unmodified.	Pass
`AddMarketOrder ToEmptyBook`	Matching Engine	Market order submitted when no opposing liquidity exists.	Order rejected or stored temporarily (depending on rule) with zero fill.	Handled gracefully without crash.	Pass
`CancelInvalid Order`	Matching Engine	`CancelOrderDeath Test` attempts to cancel a non-existent Order ID.	Contract assertion failure / death test triggered gracefully.	Process terminated via assertion as expected.	Pass
`GetInvalid OrderID`	Matching Engine	`GettersDeathTest` queries a non-existent Order ID.	Contract assertion failure.	Process terminated via assertion.	Pass

Continued on next page

Table 6: Comprehensive execution matrix of test cases across backend services and common libraries (continued)

Test Name	**Component**	**Scenario/ Description**	**Expected Result**	**Actual Result**	**Pass /Fail**
`Insufficient Balance`	Order Manager	`HasSufficient BalanceTest` checks if a trade exceeds current broker balance.	Validator returns false; order rejected before routing.	Returned false; balance remains unchanged.	Pass
`DecrementBelow Zero`	Order Manager	`UpdateBalance DeathTest` attempts to decrement balance below 0.	Contract assertion failure / death test triggered.	Process terminated via assertion.	Pass
`InvalidCancel Request`	Order Manager	Validation of an improperly formed cancel request payload.	Validator flags payload as malformed.	Rejected prior to matching engine.	Pass
`ValidMarketBid`	Order Manager	Validation of a properly formed market bid payload.	Passes semantic checks; forwards to engine.	Successfully forwarded to Matching Engine.	Pass
`ForwardAnd ReplyFails`	Order Manager	`ForwardAndReply Test` checks behavior when Matching Engine connection drops.	Order Manager gracefully handles socket failure and replies with rejection.	Rejection report sent to user.	Pass
`LoginWrong Password`	Backend Service	User attempts login with incorrect credentials.	Service returns 401 Unautho-rized.	HTTP 401 returned.	Pass

Continued on next page

Table 6: Comprehensive execution matrix of test cases across backend services and common libraries (continued)

Test Name	**Component**	**Scenario/ Description**	**Expected Result**	**Actual Result**	**Pass /Fail**
`Signup DuplicateUser`	Backend Service	Registration attempt with an existing email/username.	Service returns 409 Conflict or 400 Bad Request.	Correct error code returned.	Pass
`BasicEnqueue Dequeue`	Thread Safe Queue (Catch2)	Single-threaded enqueue and dequeue operations.	Elements are enqueued and dequeued in FIFO order.	Operations succeed and order is preserved.	Pass
`Multithreaded Producer Consumer Correctness`	Thread Safe Queue (Catch2)	Multiple producers and consumers operating concurrently.	All elements are processed without data races or deadlocks.	Data integrity maintained across threads.	Pass
`insert_ order succeeds and row is queryable`	Database Client (Catch2)	Inserting a new order record into the database.	Order is successfully inserted and can be retrieved.	Record stored and queried successfully.	Pass
`concurrent insert_ order from multiple clients`	Database Client (Catch2)	High-concurrency insertions from multiple threads.	All orders are inserted accurately without dropped transactions.	Database handles concurrent writes correctly.	Pass
`MPSC Producer/ Consumer`	Lock-free Queue (Catch2)	Single producer and consumer operating over a lock-free queue.	Messages are safely passed without locks.	High-throughput message passing achieved.	Pass
`Client Server Ping Pong`	Network/ WebSocket (Catch2)	Bi-directional message echoing between client and server.	Messages are received intact and in order.	Successful real-time communica-tion.	Pass

Continued on next page

Table 6: Comprehensive execution matrix of test cases across backend services and common libraries (continued)

Test Name	**Component**	**Scenario/ Description**	**Expected Result**	**Actual Result**	**Pass /Fail**
`Two Client One Server`	Network/ WebSocket (Catch2)	Multiple clients connecting and communicating with a single server concurrently.	Server correctly identifies and routes messages to respective clients.	Multiplexed communication successful.	Pass

6.6 Simulation Default Configuration

The following tables outline the complete default configuration values used to initialize the simulation platform's agents and the Oracle Service.

Component	Parameter Name	Default Value
Noise Trader	`num_noise_traders`	3
	`lambda_eps`	0.6
	`bernoulli`	0.5
	`pareto_scale`	0.5
	`pareto_shape`	4.0
	`initial_inventory`	20.0
Market Maker	`num_market_makers`	4
	`lot_size`	1.0
	`gamma`	0.6
	`k`	8.0
	`terminal_time`	30.0
	`initial_inventory`	300.0
Informed Trader	`num_informed_traders`	1
	`initial_inventory`	100.0
	`epsilon`	0.05
	`trade_qty`	10.0
	`max_inventory`	1000.0
Oracle Service	`mu`	0.0
	`sigma`	0.003
	`jump_intensity`	0.06
	`jump_mean`	0.0
	`jump_std`	0.01
	`update_interval_ms`	60000

Table 7: Complete default configuration parameters for the simulation components.